React App Explorer: redesign brief

This brief is for a designer (Claude Design) doing a complete redesign of React App Explorer. It covers what the app does, every screen and state as it looks today, the visual language, what must not change, the technical limits, and what to deliver.

Everything a redesign needs is in this one document: the brief, every screen and state (section 3, full-width screenshots from the running app, 1440 × 900), and appendices with today's design tokens (A), the diagram's CSS (B), the logo artwork (C) and a sample of the data the app shows (D).

---

1. The product

React App Explorer maps a React or Next.js codebase as an interactive diagram. A scanner reads the source, using oxc to parse it and TypeScript 7 for types. It writes a *snapshot*: every file, component, hook, function, server action, TypeScript type and Next.js route, and how they relate. The viewer lets you explore that snapshot one step at a time:

- add an element;
- expand its parents or children;
- read its props, types, render tree and caching;
- open the source in your editor.
- **Who uses it:** front-end developers and tech leads who want to understand a large or unfamiliar app. They are technical and keyboard-friendly, and they keep the app open next to their editor.
- **Where it runs:**
    - As a **web app** in Chrome, served by Vite.
    - As a **desktop app** (Tauri, macOS first). It shows the same UI in WebKit (Safari's engine), with no browser toolbar.
    - Both must look and behave the same. See §7.
- **How a session goes:**
    1. Scan a project (or pick an earlier snapshot).
    2. The diagram starts empty: add an element from the file tree or the picker.
    3. Grow the diagram along relations (↑ parents, ↓ children, right-click menus).
    4. Read the details on the right; jump to the code.
    5. Come back later: the diagram, selection and settings are kept.

2. The model: what appears on screen

Node kinds (each has its own colour and icon; the colours carry meaning)

KIND	MEANING	COLOUR TODAY	ICON (LUCIDE)
`module`	A source file	gray	Box
`component`	React component	**#429cb5** (custom teal, "component" palette)	CodeXml `</>`
`hook`	React hook	purple	Anchor
`serverAction`	`"use server"` function	orange	Zap
`function`	Plain function	teal	FunctionSquare
`type`	TypeScript type / interface	green	Braces `{}`
`route`	Next.js App Router route	pink	Signpost
`external`	npm package (library)	yellow	Package
folder	A folder (added explicitly; not a scanned kind)	neutral	Folder

Extra states on a node:

- **Boundary:** `server`, `client` or `shared` chip (gray, cyan and pink).
- **Next.js role:** page, layout, loading, error, route, middleware…
- **Flags:** `async`, missing file (a broken import), asset (CSS or images; hidden by default).
- **Routes:** rendering badges ○ Static, ◐ Partial, ƒ Dynamic, ⚠ Blocking, with revalidate times. The reasons appear in the details.
- **Cached** (`"use cache"`): "cached 1h" in the caption.

Relations (edges): each has a colour, a word, and sometimes a dash

RELATION	WORD ON THE EDGE	COLOUR TODAY
imports	"imports" (dashed: lazy or dynamic import; muted: type-only)	dark gray
declares	"declares" (a file → what's in it)	gray
renders	"renders" (can carry *conditions*, e.g. `when user && isAdmin`)	component teal
usesHook	"uses hook"	purple
invokes / invokesAction	"invokes" / "server action"	teal / orange
usesType	"uses type"	green
navigates	"links to" (Link, router.push, redirect)	pink
fetches	"calls API"	cyan
revalidates / invalidates	"revalidates" / "invalidates"	gray / purple

RELATION	WORD ON THE EDGE	COLOUR TODAY
routeFile / routeChild	"route file" / "nested route"	light pink
runsBefore	"runs before" (middleware → route)	red

Several relations between the same two nodes are drawn side by side in *lanes*, with their labels stacked one under another.

Containment

A **file** that has elements drawn inside it is a **frame** around them: header, chips and actions on top, cards inside. Folders and routes are separate nodes, never automatic frames.

---

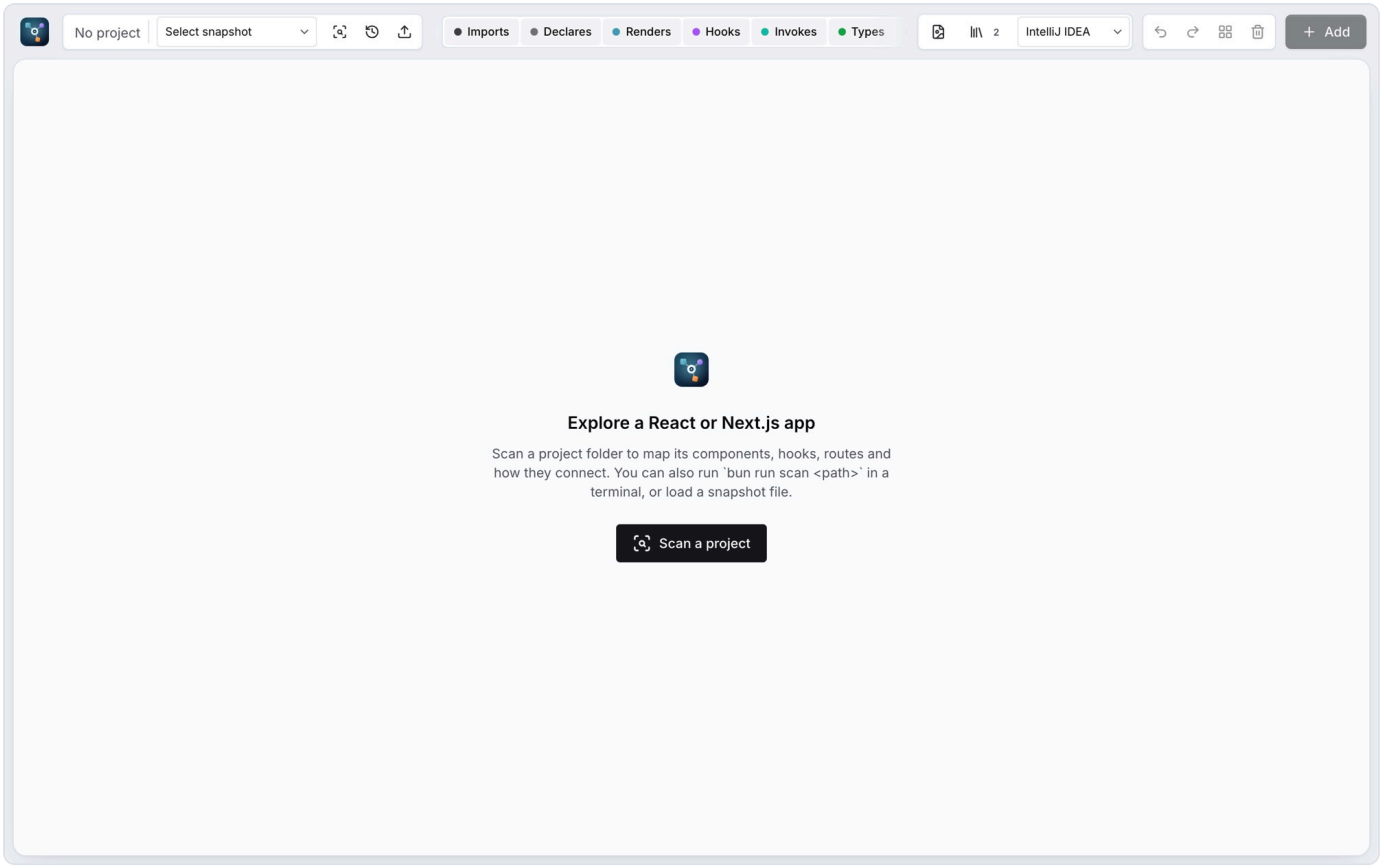
3. Screens and states (screenshots follow the table)

#	SCREEN / STATE	NOTES
01	Welcome: no snapshot	Brand, one call to action (Scan a project)
02	Snapshot loaded, empty diagram	"Empty diagram" prompt, the file tree, the round + button
03	A component selected	The app's main state: card in its file frame, children, edges with labels, details panel
04	Node tooltip on hover	Props, hooks, types; shows after 600 ms
05	Right-click menu on a node	Actions, then "Relations" submenus listing related elements
06	Add picker (Space / ⌘K / +)	Tree with checkboxes, filter, "Add all", "Done"
07	Quick add (right-click on empty canvas)	Adds at that point
08	Details: hook	Inputs & output, uses / used by
09	Details: TypeScript type	Generics, definition (code), uses
10	Details: Next.js route	URL, params, rendering badge and reasons, files
11	Details: server action	
12	Details: file (frame)	Exports, imports
13	Details: library	
14	Details: folder, picked in the file tree	Not on the diagram: "Add to diagram" comes first
15	Route chain (layouts → page) with navigation	Pink route cards, files, "links to"
16	Direct children placed as a grid	Placement behaviour; the look of a dense diagram
17	Scan dialog	Folder browser, path input, types on/off
18	Scan progress	Floating card: files scanned, then types, with Cancel
19	Snapshots dialog	Grouped by scanned folder; delete

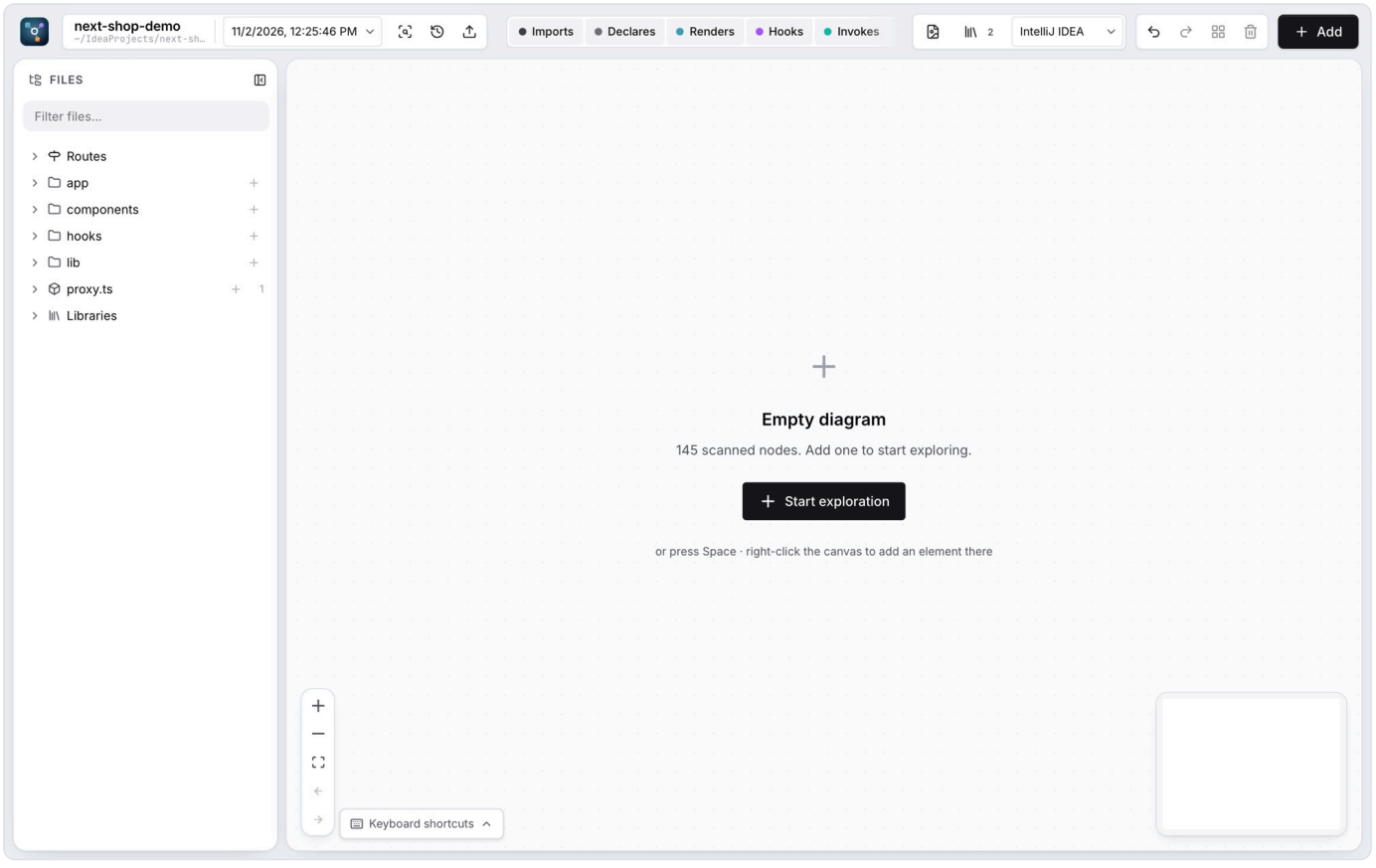
#	SCREEN / STATE	NOTES
20	Libraries dialog	Which npm packages are hidden
21	Keyboard shortcuts legend	Bottom-left, expandable
22	Both side panels minimized	File tree and details folded to strips (animated)
23	Confirm a large expansion	"Add 140 elements?"
24	Notice (toast)	e.g. the editor couldn't open
25	A big diagram (60 elements)	Stress test for legibility

States not captured, but they need designs too:

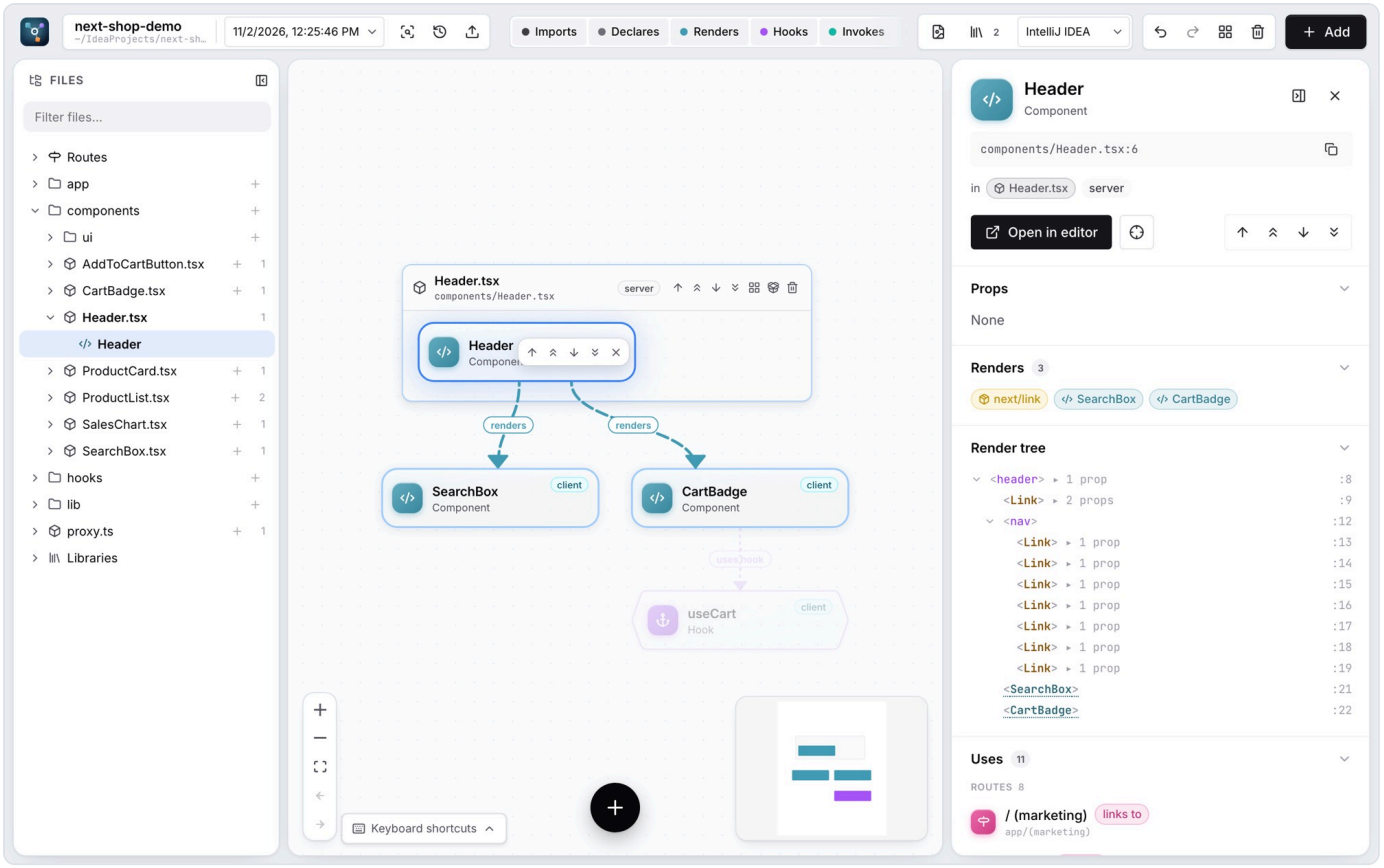
- Hover, focus-visible, pressed and disabled for every control. Selected versus *related* (highlighted) versus faded nodes: when something is selected, everything unrelated fades to 32%.
- Dragging a node (frames grow and push neighbours). Dragging the panel resize handles.
- Scan **failed**, **cancelled**, "diagram ready **without types**" (with message), loading a snapshot (spinner).
- A file-tree filter with no matches. A long tree that's cut off ("refine the filter").
- Edge **conditions** (`when …`) and "+N variants" labels. Dashed and animated edges for lazy imports.
- A missing file (broken import) node. Asset nodes (when shown).
- Desktop app: the loading screen while the local server starts, and the failure page when it can't start (`packages/desktop/loading/index.html`).
- The toolbar on narrow windows (1280 px): the relation filters scroll sideways and fade at their end.
- Dark mode: dark values exist for most tokens but have never been designed. This is a decision to make (§9).



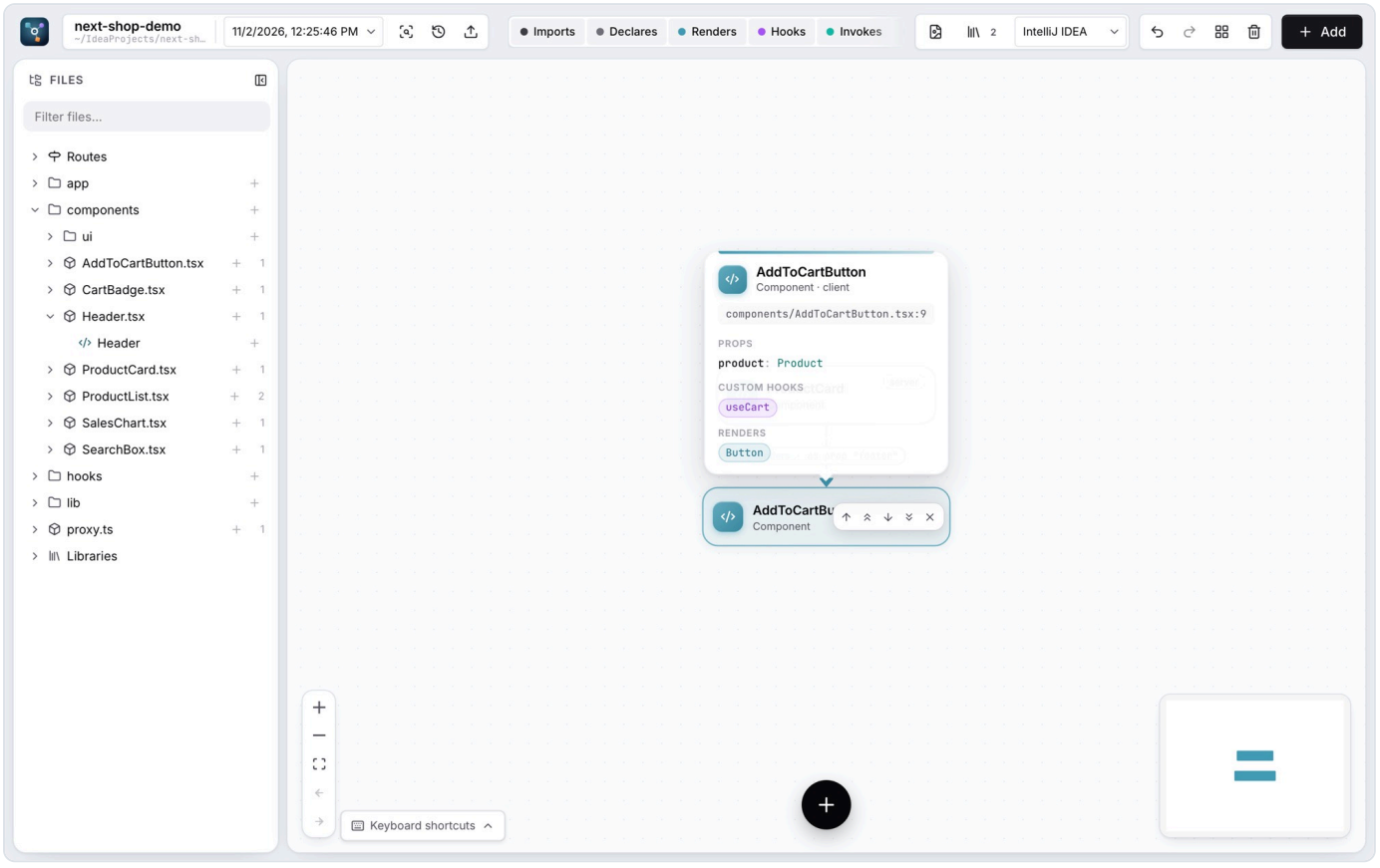
01 · Welcome No Snapshot



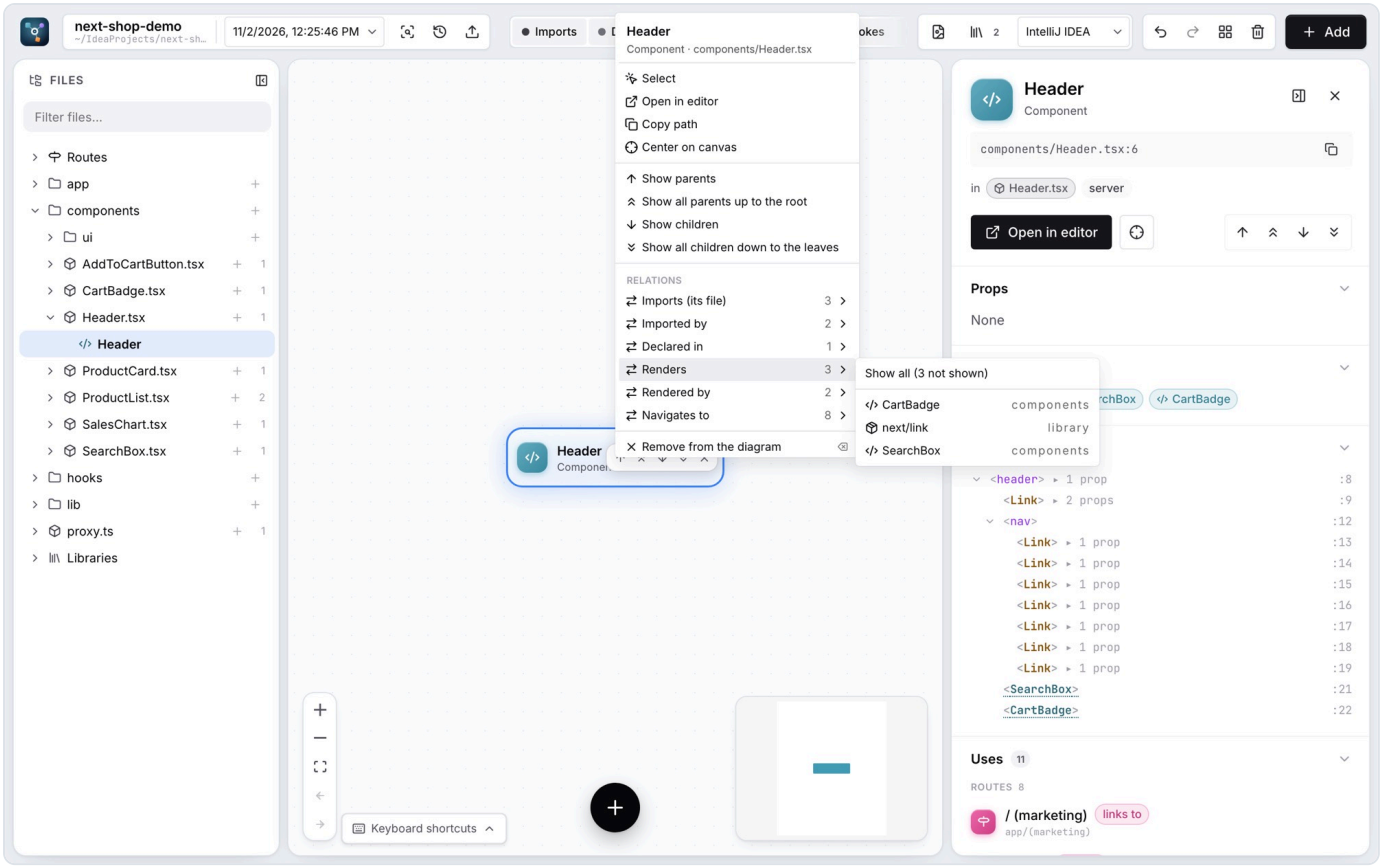
02 · Empty Diagram



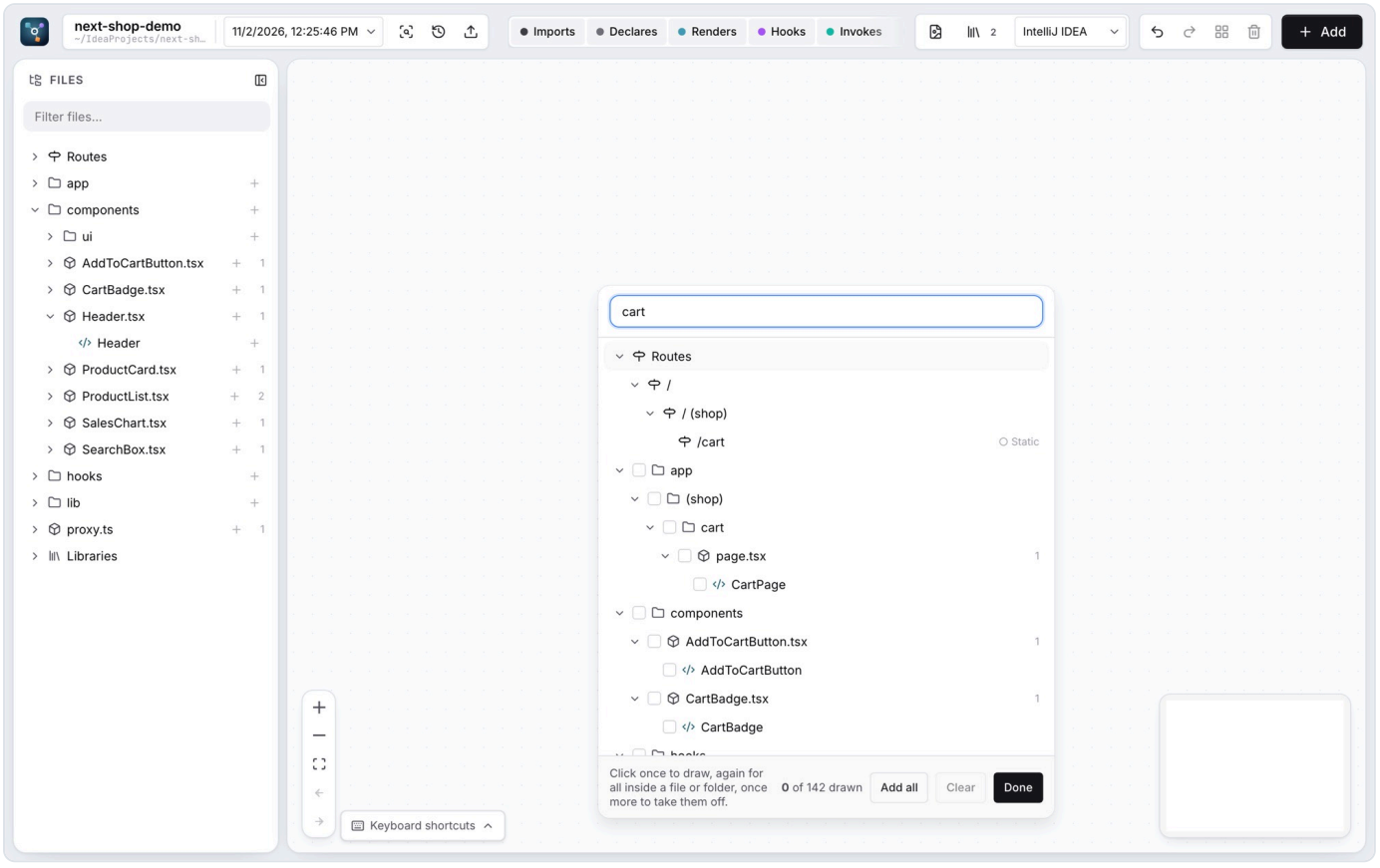
03 · Component Selected



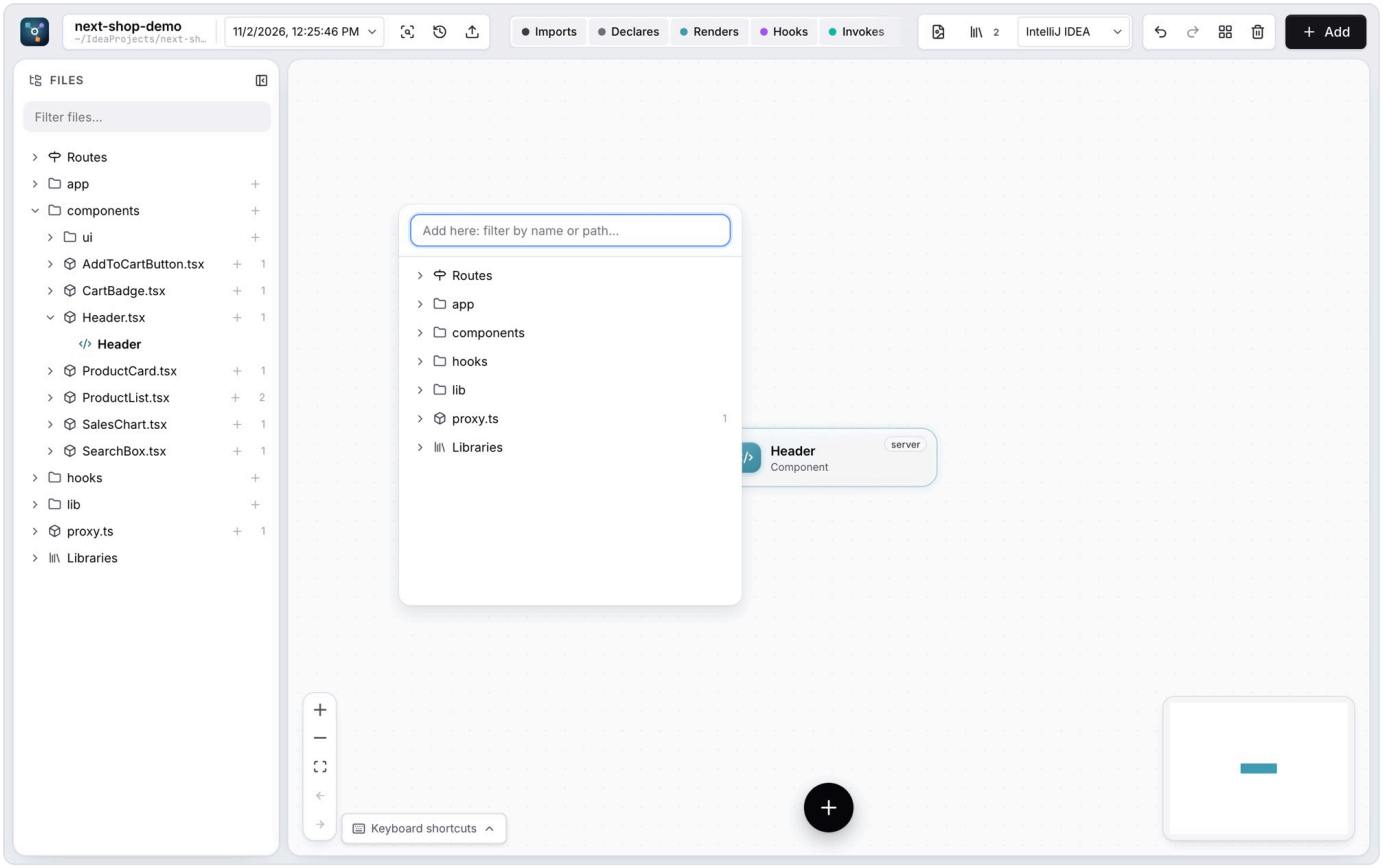
04 · Node Tooltip



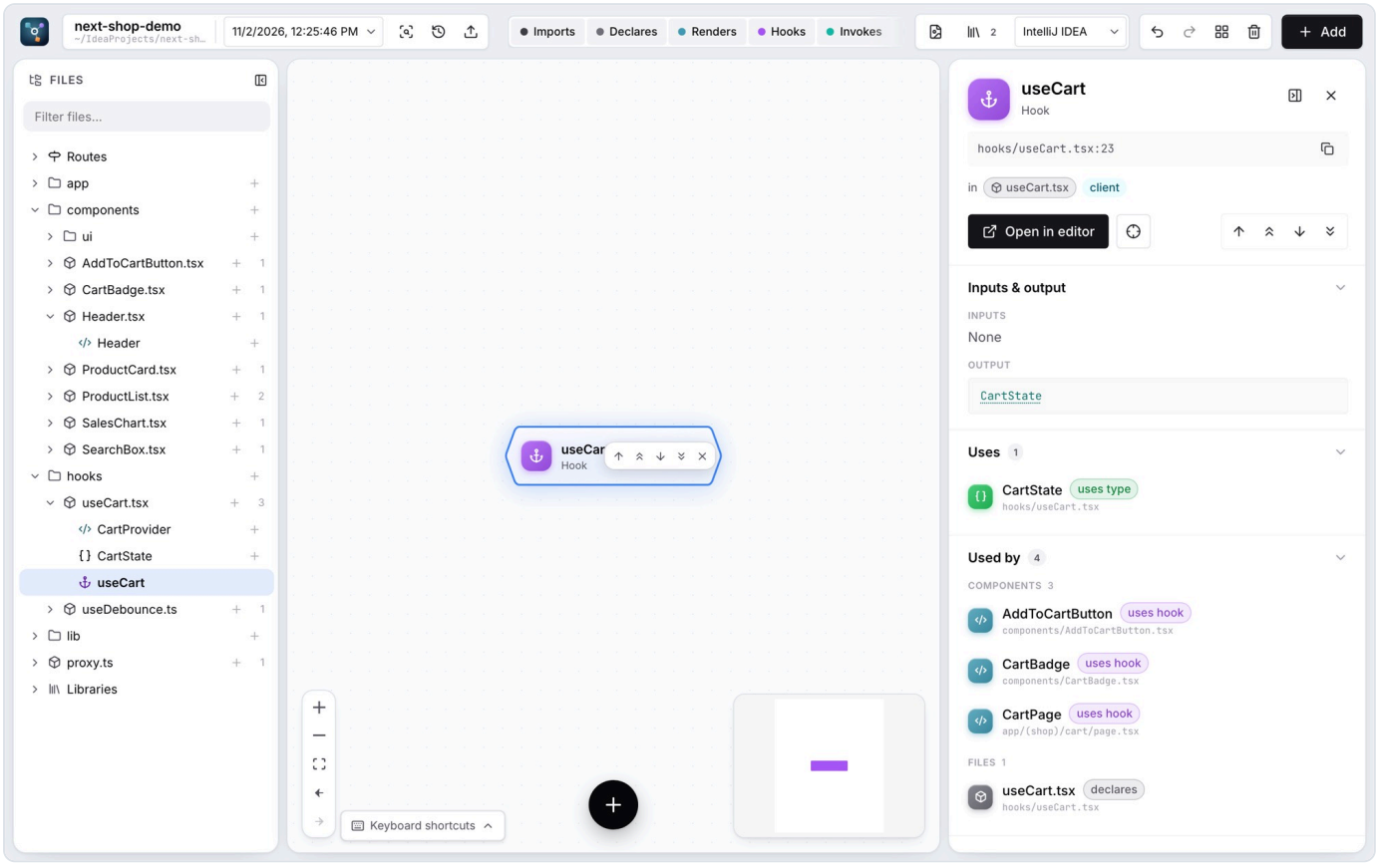
05 · Context Menu



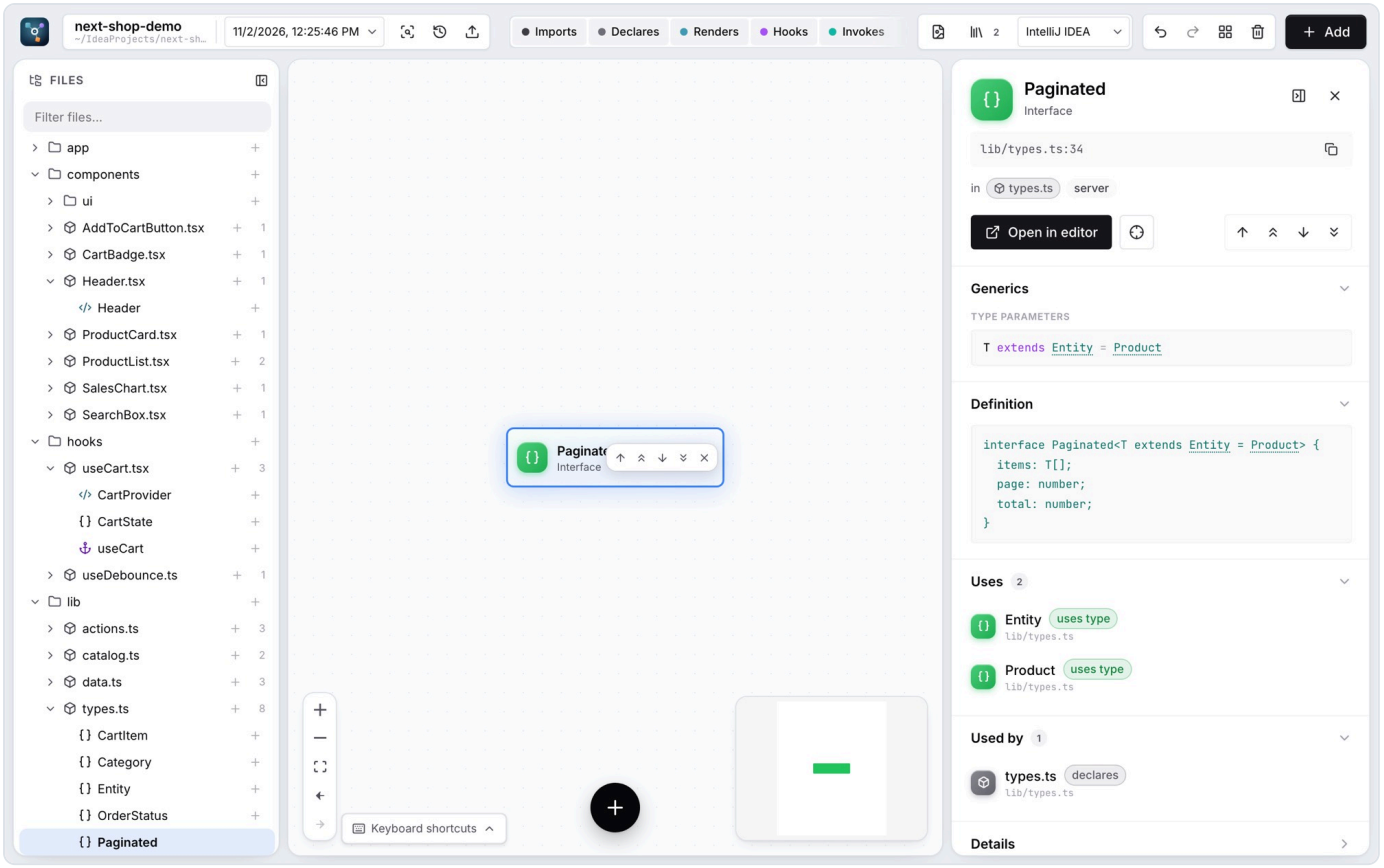
06 · Add Picker



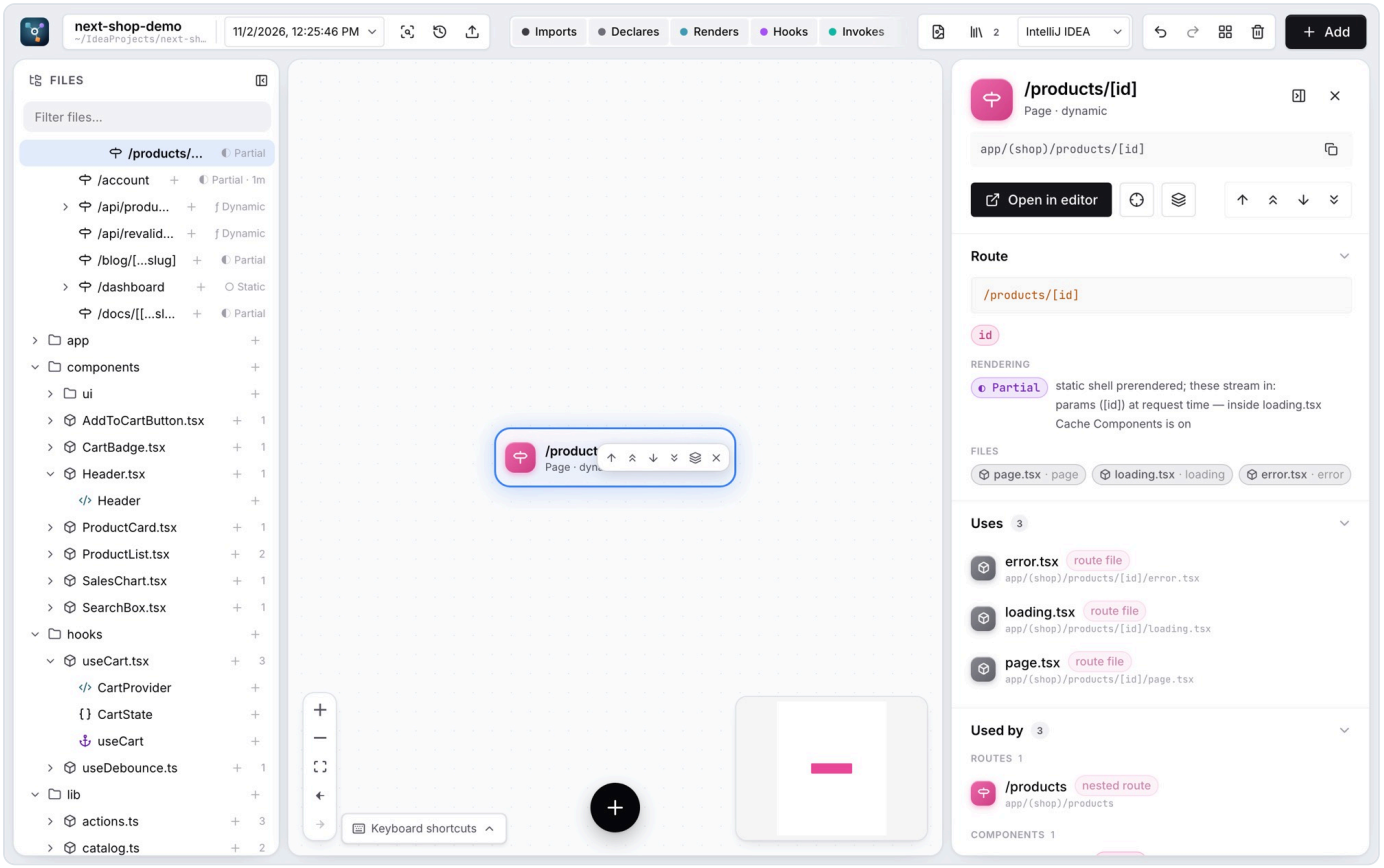
07 · Quick Add



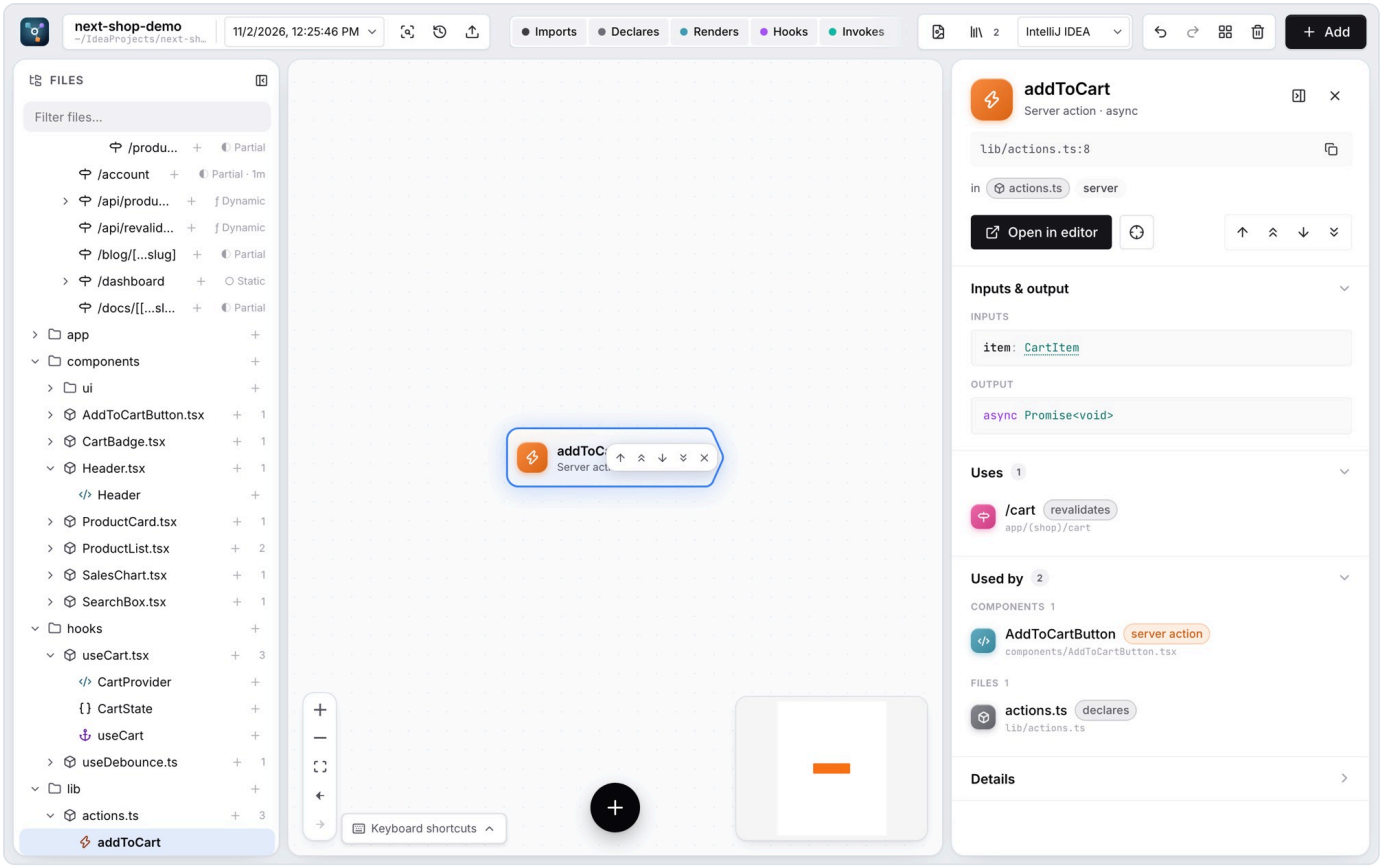
08 · Details Hook



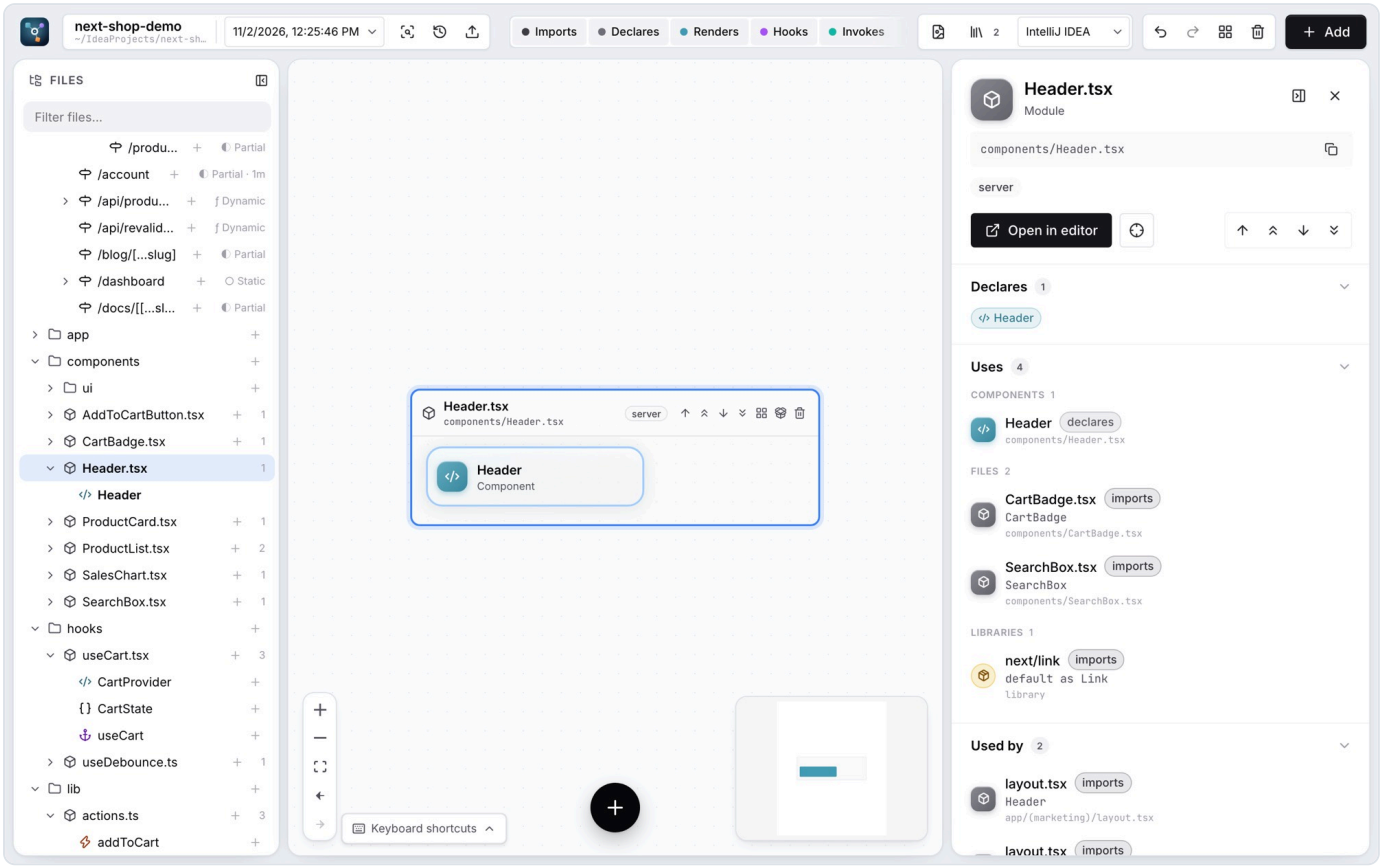
09 · Details Type



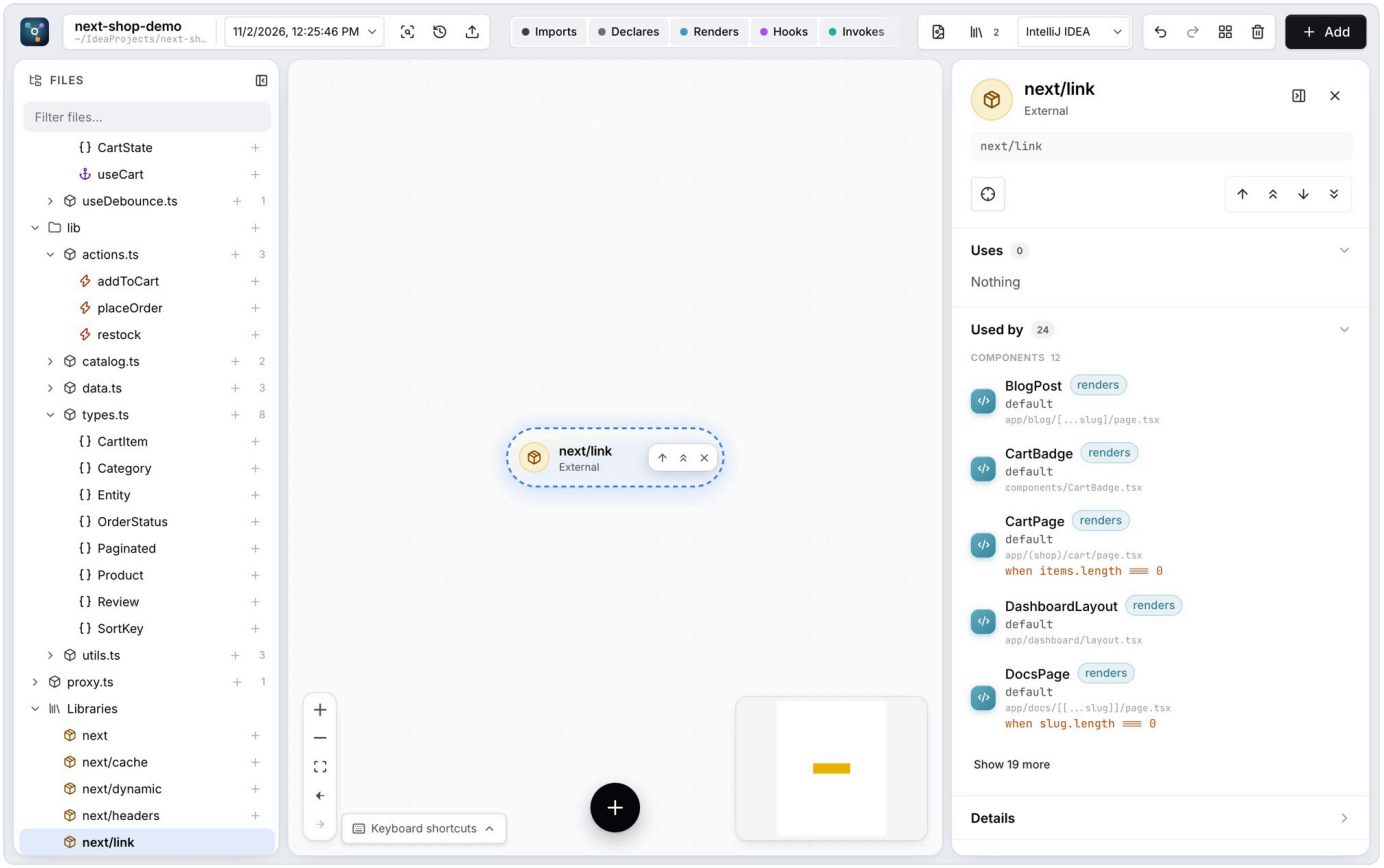
10 · Details Route



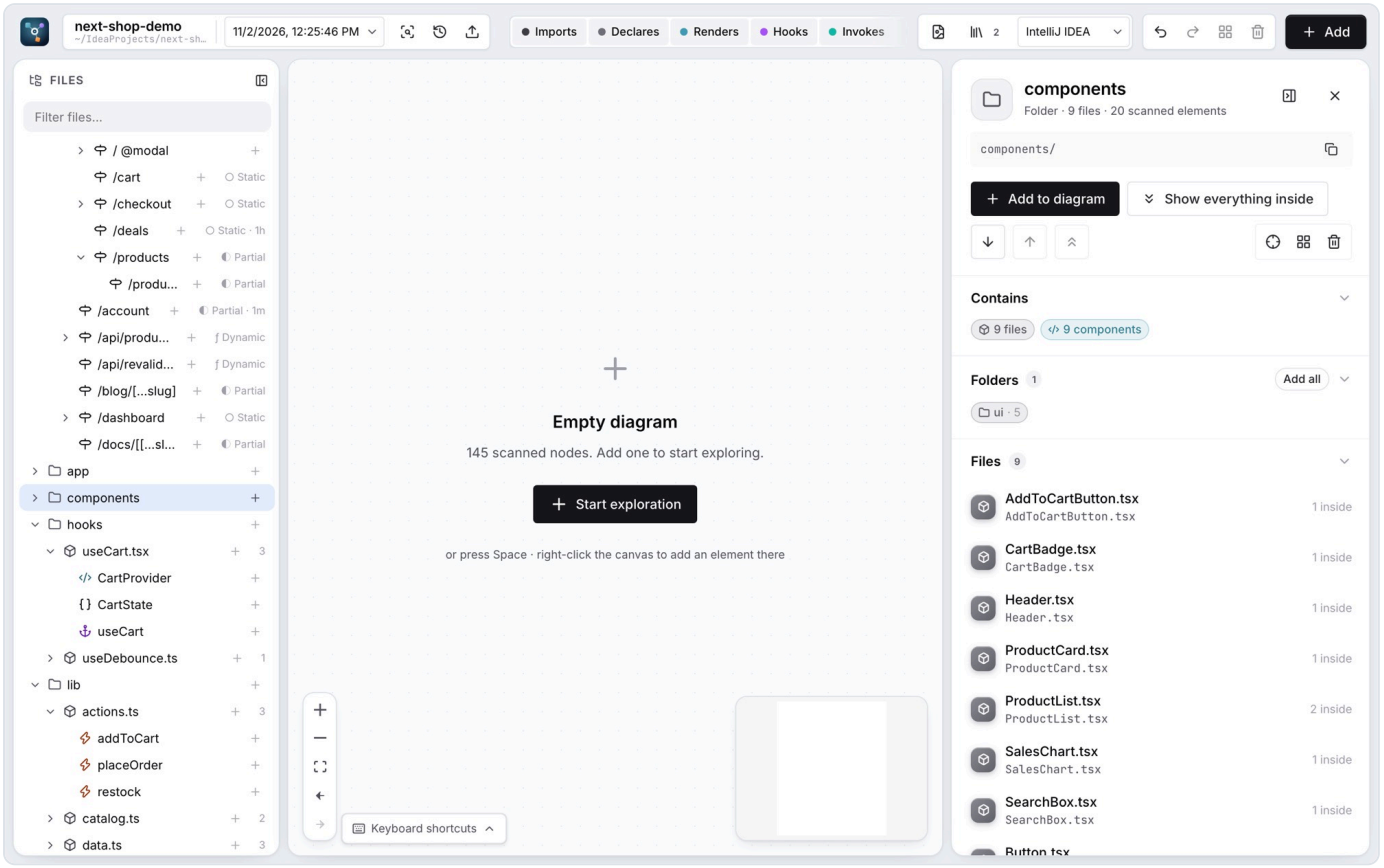
11 · Details Server Action



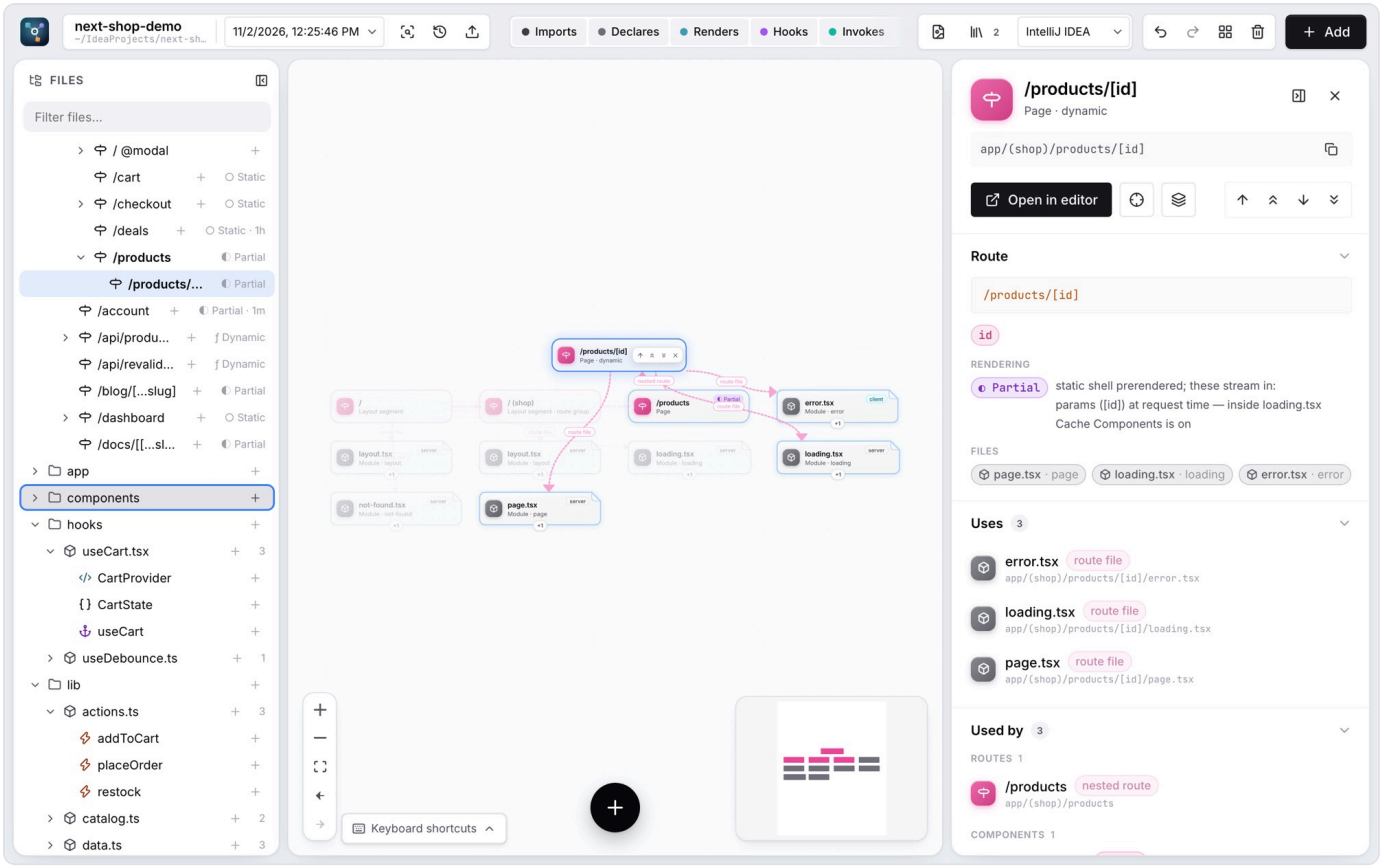
12 · Details File Frame



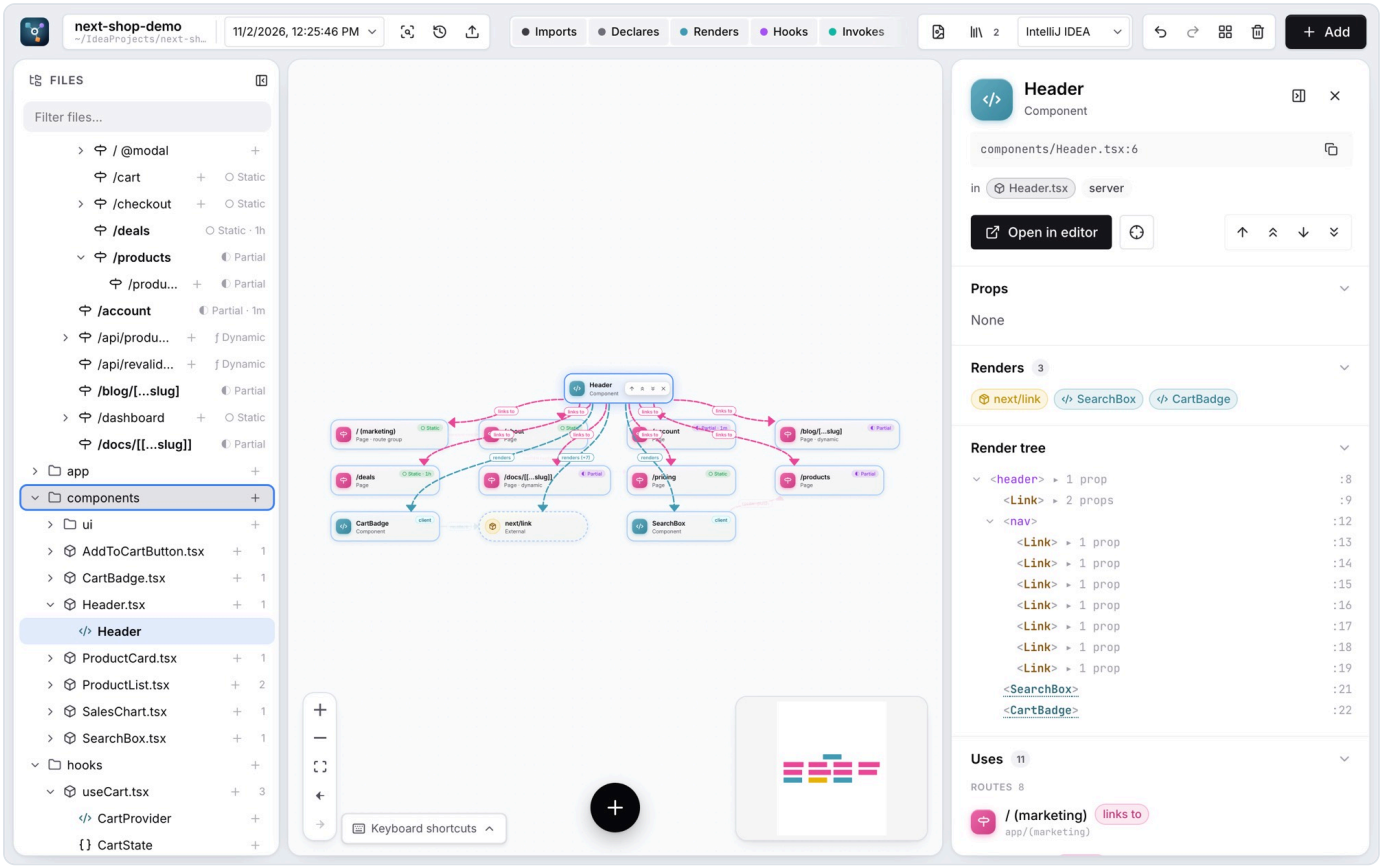
13 · Details Library



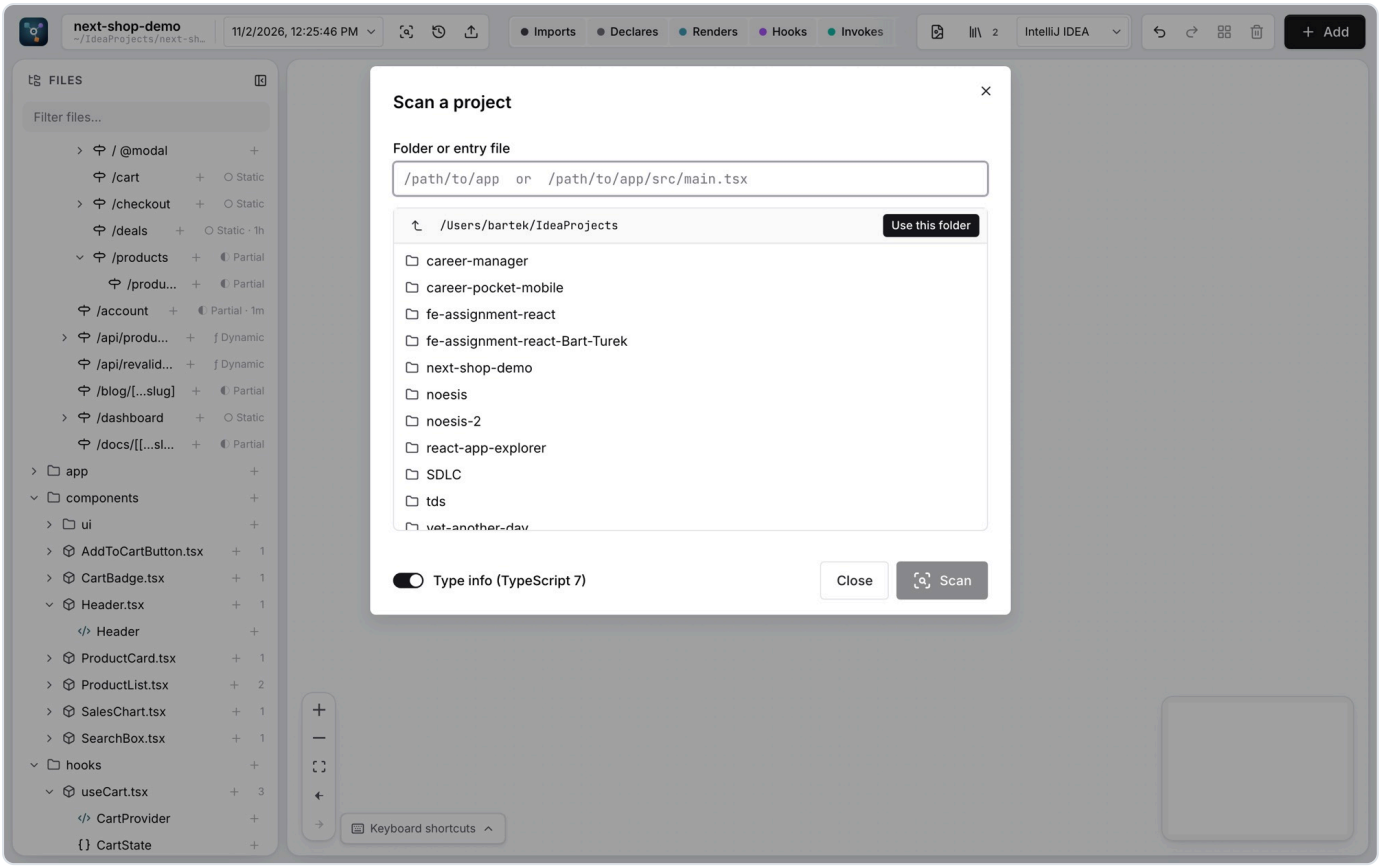
14 · Details Folder Inspected From Tree



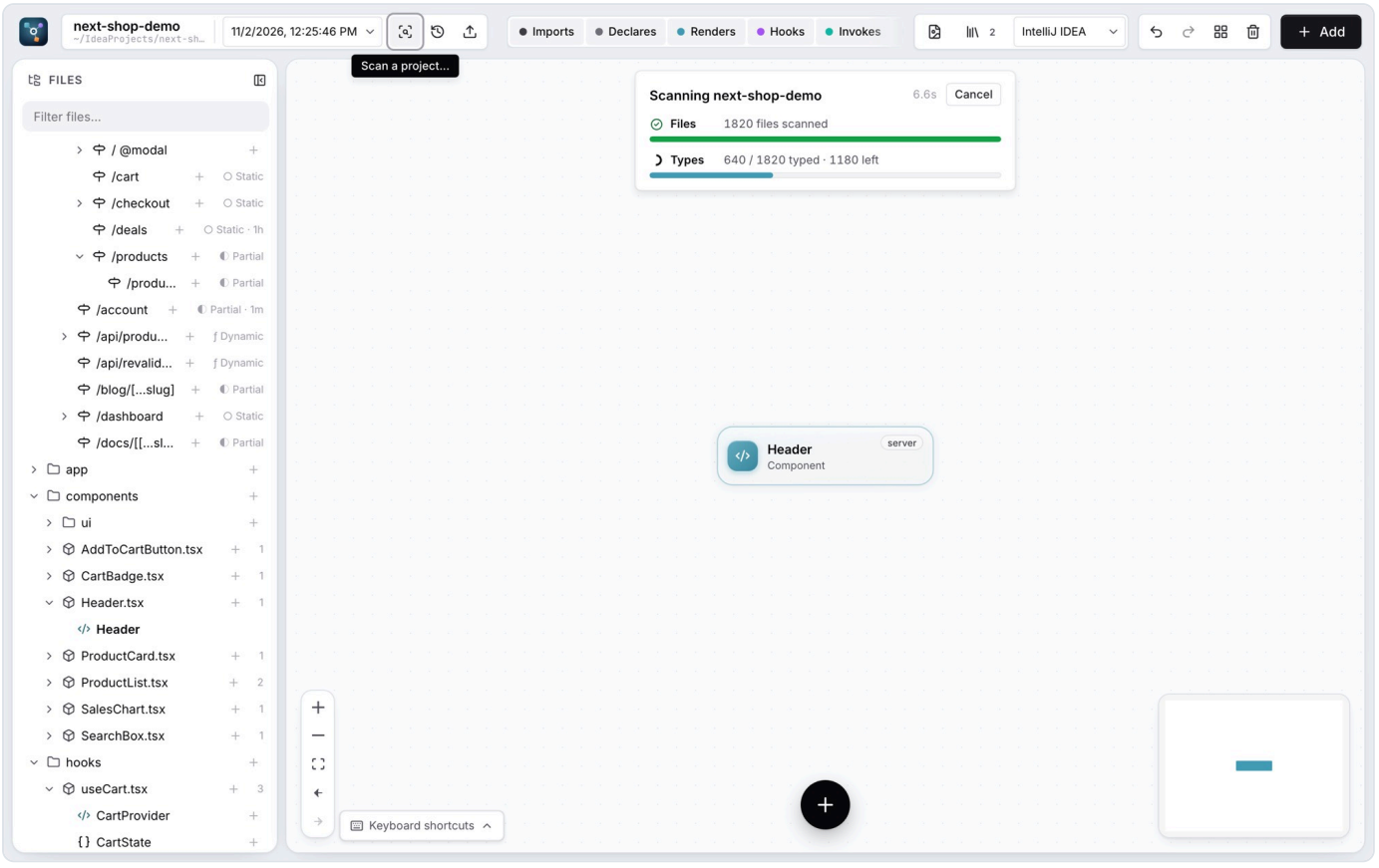
15 · Routes And Navigation



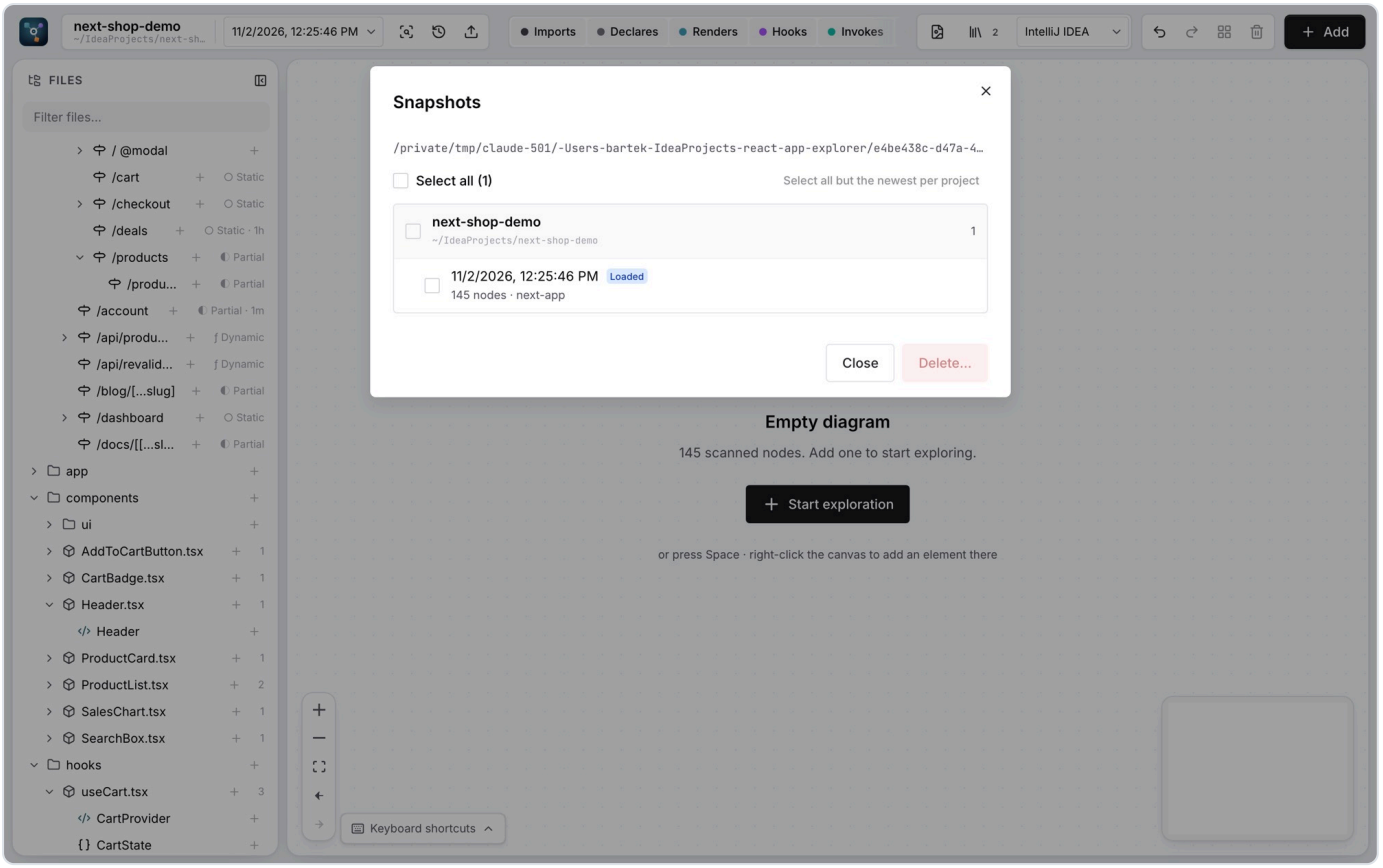
16 · Children Grid



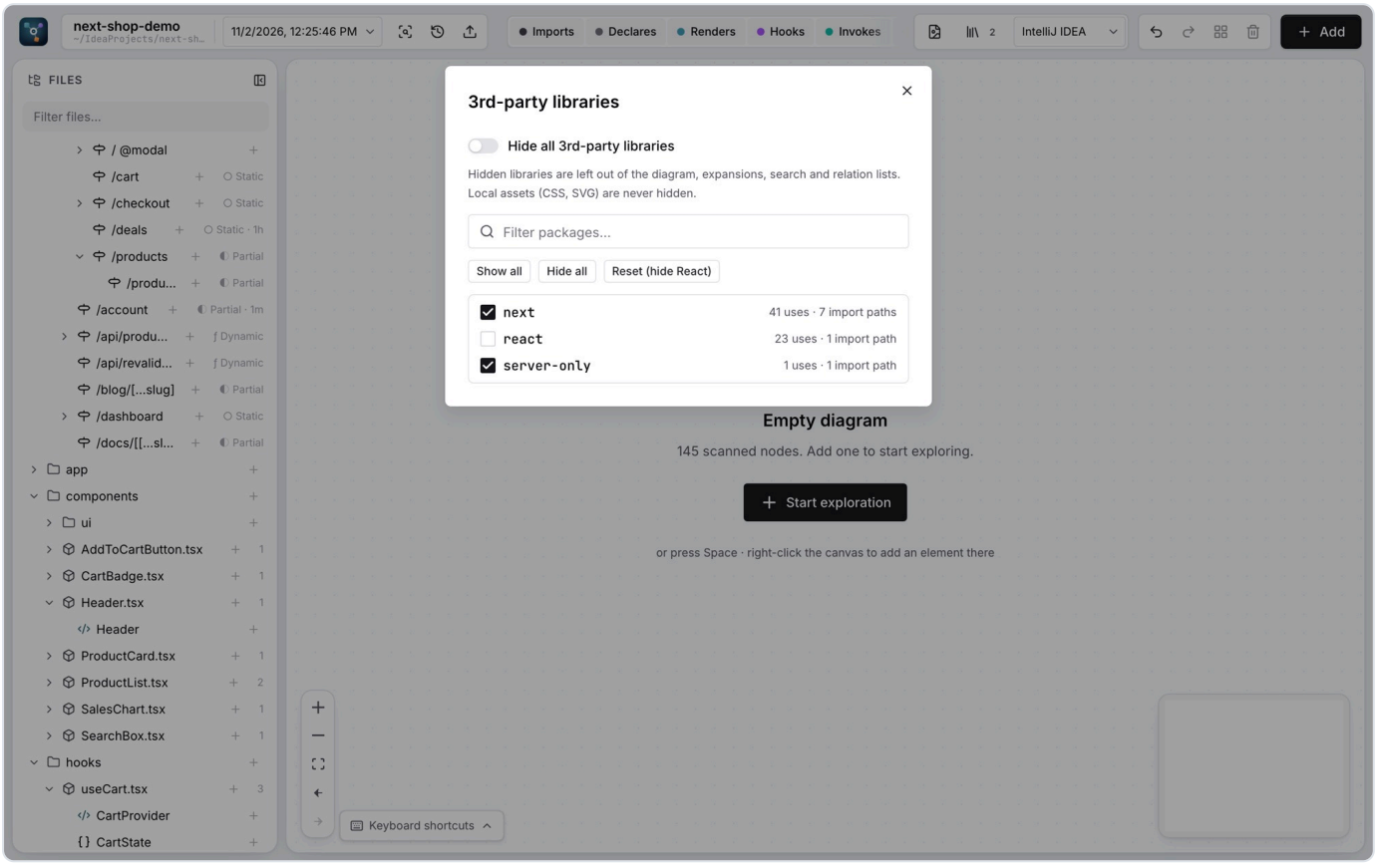
17 · Scan Dialog



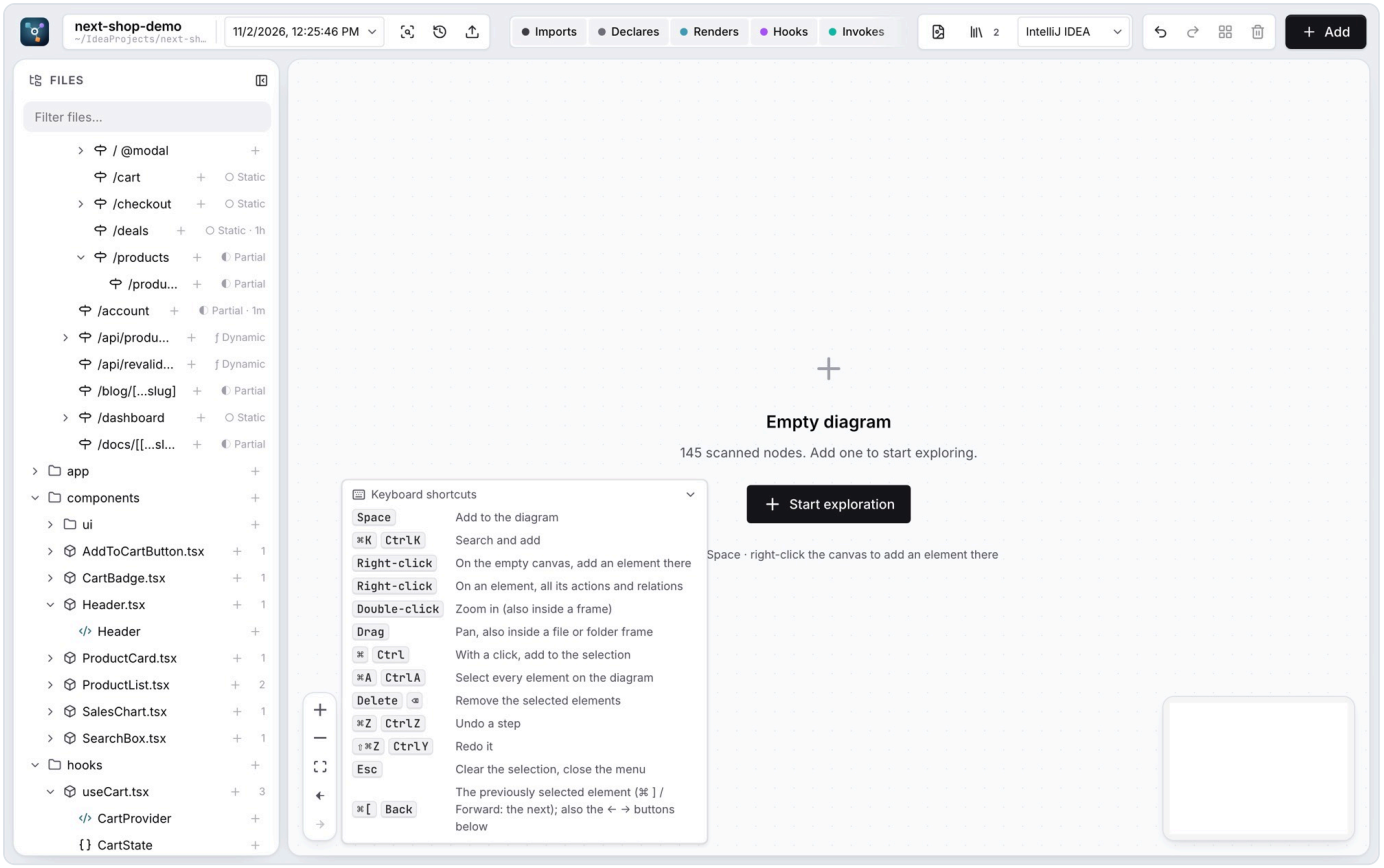
18 · Scan Progress



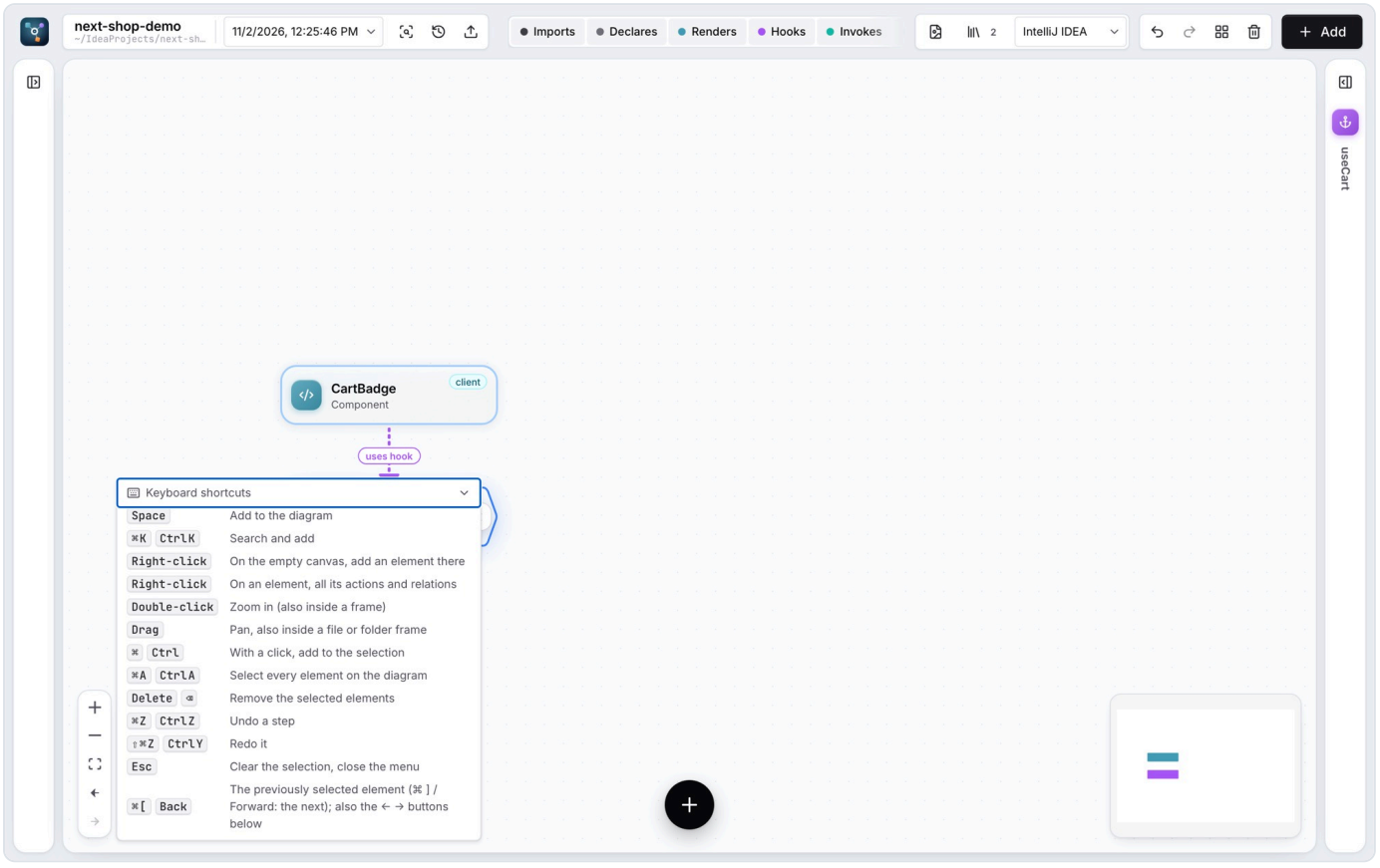
19 · Snapshots Dialog



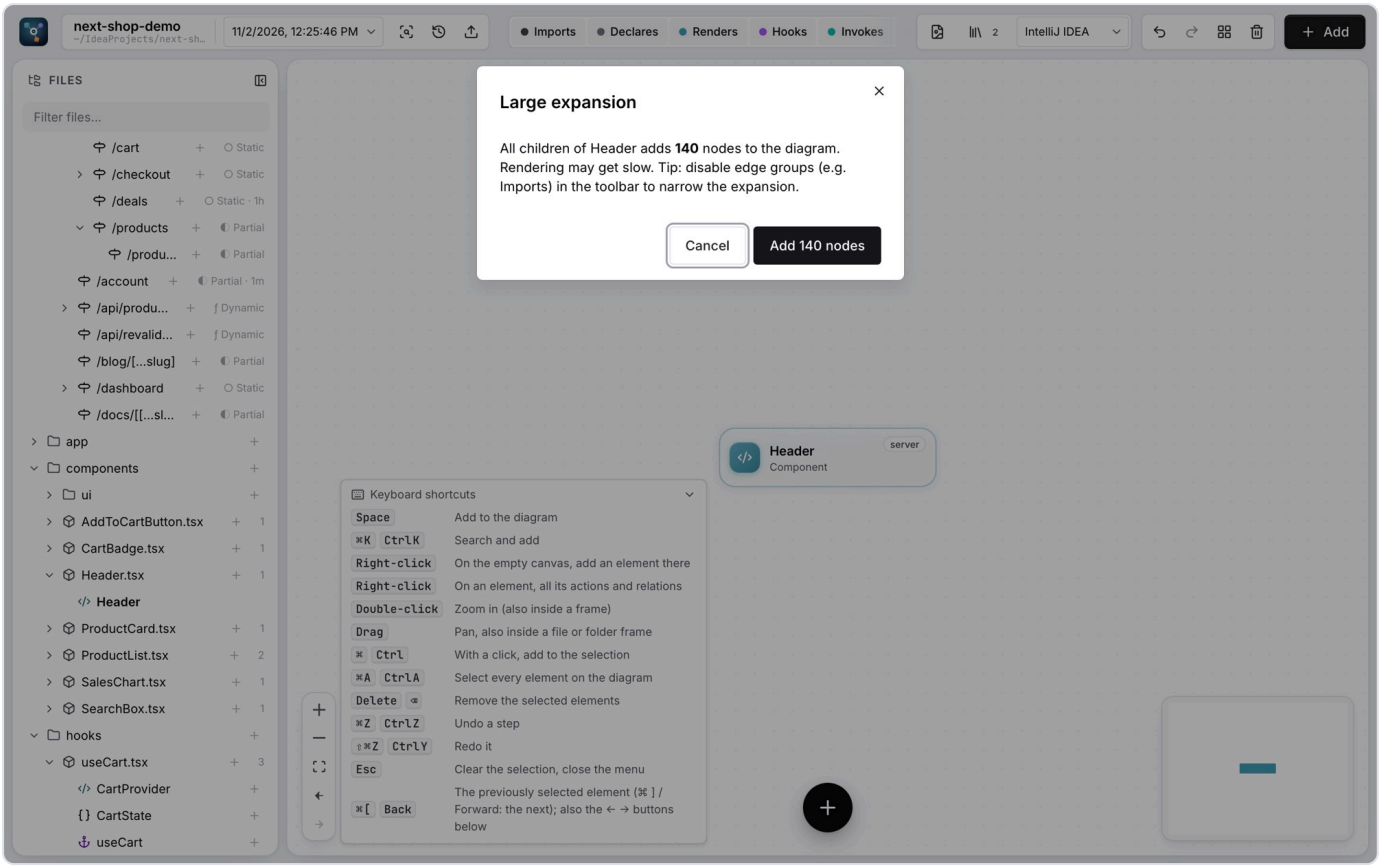
20 · Libraries Dialog



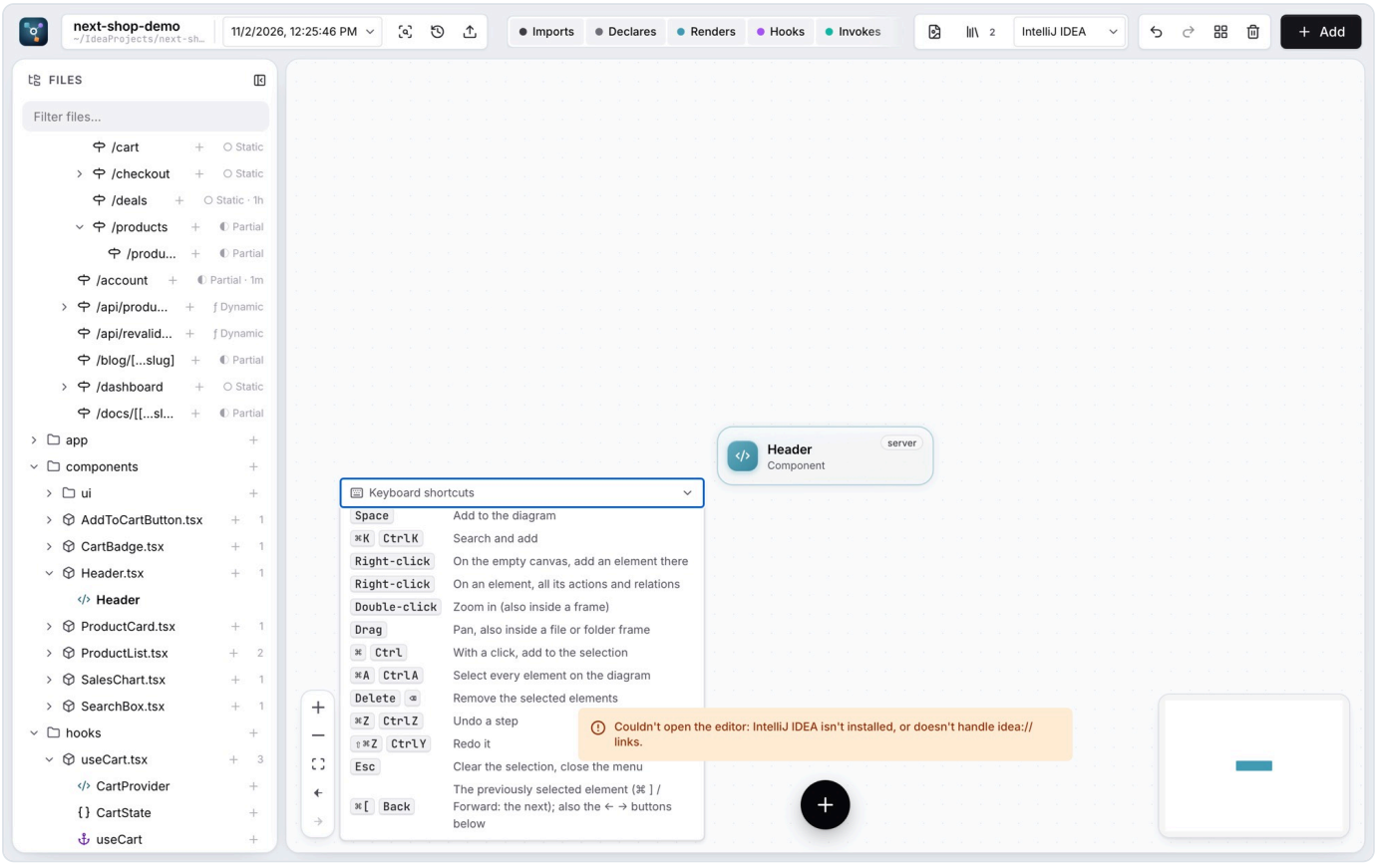
21 · Keyboard Shortcuts



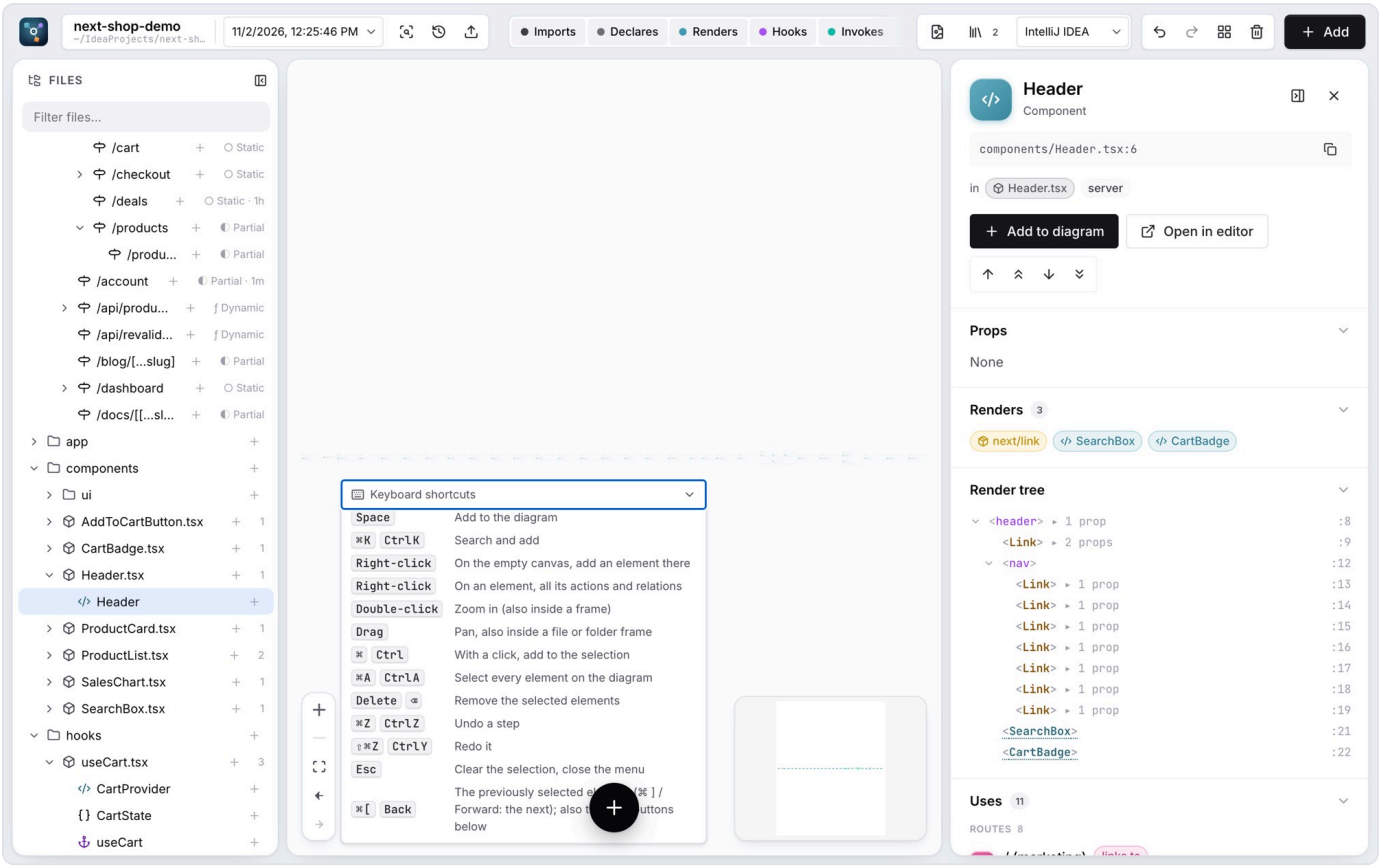
22 · Panels Minimized



23 · Confirm Large Expansion



24 · Notice



25 · Big Diagram

---

4. Layout and interaction model (today)

```
┌──────────────────────────────────────────────────────────────────────────────┐
│ [mark] [project name · path | snapshot ▾ | scan · snapshots · load] | relation filters | view | history | [+ Add] |
├──────────────┬──────────────────────────────────────────────┬────────────────┤
│ FILES        │                canvas (diagram)              │ details panel  │
│ filter       │                                              │ (selected or   │
│ tree +       │  zoom · fit · ← →        (+)        minimap  │  inspected     │
│ (+ to add)   │  keyboard shortcuts                          │  element)      │
└──────────────┴──────────────────────────────────────────────┴────────────────┘
```

- **Shell:** the panels float on a grey backdrop with 8 px gaps (14 px radius, soft shadow).
  - **File tree** (left): resizable 200–560 px; folds to a 44 px strip.
  - **Details** (right): resizable, 440 px by default; minimizes to a strip showing the element's icon and name.
  - **Canvas:** fills the middle.
- **Toolbar groups,** left to right:
  - brand;
  - **workspace** (project and snapshot, scan, manage snapshots, load file);
  - **relation filters** (8 colour-dot toggles);
  - **view** (assets, libraries filter, editor choice);
  - **history** (undo, redo, re-layout, clear);
  - **primary:** + Add.

- **Canvas controls:** zoom in/out, fit, Back/Forward (selection history), the minimap, the round "+" add button, and the keyboard shortcuts legend.
- **Per-node actions** (appear on hover or selection): ↑ parents, ⇈ up to root, ↓ children, ⇊ down to leaves, × remove. Frames add: arrange, unwrap and remove-with-contents.
- **Keyboard:**
  - Space / ⌘K: add.
  - ⌘A: select all.
  - Delete: remove.
  - ⌘Z / ⇧⌘Z: undo / redo.
  - Esc: clear the selection.
  - ⌘[/ ⌘]: back / forward.
- **Persistence:** the diagram per project, the selection, panel widths and open states, filters, editor choice.

---

5. Visual language today (starting point, not a constraint)

- **Type:** Inter Variable (UI) and JetBrains Mono Variable (code), both bundled.
- **Radii:** 6 / 8 / 12 px, and 14 px for panels.
- **Colour:** cool greys for the shell (`#ECEFF3`); white surfaces; a near-white canvas with a dot grid.
- **Cards:** "glass" cards with a light tint of the kind colour from the top-left corner, a gradient icon tile per kind, and a thin border that turns blue when selected. Each kind has its own card shape: hooks are hexagons, external libraries have a dashed outline, files have a folded corner.
- **Brand:** the mark shows a component (teal square), a hook (violet circle) and a server action (amber tile) linked to the explored node, on a deep teal-to-ink tile. Feel free to rework it.
- **Selection is the only use of blue.**

---

6. What must stay (product rules)

1. **The kind colours and relation colours encode meaning.** The same colour must be used for a kind's card, its tree icon, its details tile and its chips, and for a relation's edge, label, toolbar dot and details badge. Refine the palette freely, but keep it one colour per kind and one per relation, and keep them telling apart for colour-blind users (consider a second cue: icon or shape).
2. **Selection stays unambiguous:** one accent colour is reserved for selected and related elements.
3. **Dense diagrams must stay readable:** 50–300 nodes, long names, many labels.
4. **Code is code:** monospace and syntax colours for types, props and paths (name, type, literal, keyword, punctuation).
5. **Accessible:** every icon button has a label and tooltip, there are visible focus rings, it works with the keyboard, and text contrast is at least 4.5:1.

7. Technical constraints the design must respect

- **Two engines:** Chrome (web) and WebKit (desktop). In WebKit, animating `opacity`, `transform` or `filter` on diagram nodes makes them blurry under zoom while the animation runs, so **node hover and selection changes must not depend on those transitions**; instant changes are fine. Panel and dialog animations are fine. `backdrop-filter` works but is costly over the canvas.
- **The desktop app has no browser UI:** no Back button, address bar or `alert()`. Anything users need must be in the app (the ← → buttons exist for this reason).
- **Performance:**
  - Diagram nodes and edges are plain HTML, CSS and SVG, re-rendered while panning and zooming with hundreds on screen. No heavy shadows per node, no blur per node, no web fonts besides the two bundled ones.
  - The chrome (toolbar, panels, dialogs) uses Chakra UI v3 through `@rae/design-system`, the only package allowed to import Chakra or icons.
- **Window sizes:** 1280–2560 px wide, and it must work at 1280 × 800. The toolbar currently drops labels to icons below 2080 px.
- **Icons:** lucide (any icon from the set). Custom SVG is fine for the brand and for kind glyphs.
- **Tokens:** they become Chakra semantic tokens (light and dark), e.g. `kind.hook.solid / subtle / muted / fg`, `relation.renders`, `syntax.type`, `shell.bg`. Keep the token structure, or propose a better one, mapped one to one.

8. What to deliver

1. **Direction:** two or three visual directions (moodboard and one hero screen each, state 03), before going wide.
2. **Design system:**
   - colour tokens, light and dark (neutrals, accent or selection, the 8 kind colours with solid, subtle, muted and fg, 12+ relation colours, boundary chips, syntax);
   - type scale; spacing scale; radii; shadows or elevation;
   - focus and interaction states;
   - motion (durations and easing; which elements may animate, given §7).
3. **Components** (with states):
   - toolbar groups and controls;
   - segmented toggles; select; icon buttons; buttons; chips (kind, boundary, tag, relation);
   - panel and panel header and sections; the panel strip (minimized);
   - tree rows (file tree and picker, with ticks); context menu with submenus;
   - modal; tooltip; toast; progress card; empty state;
   - code rows, inline types and type links.
4. **Diagram spec:**
   - a card for each kind (default, hover, selected, related, faded, dragging), including the route rendering badges and "cached" captions;

    - the file frame (header and room);
    - folder and route cards;
    - edges for each relation: stroke, dash, arrowheads, labels, condition labels, lanes;
    - the minimap; the canvas background; the node action toolbar.
5. **Screens:** every state in §3, including the states not captured, at 1440 × 900, plus the toolbar at 1280 and 1920.
6. **Brand:** a refined mark, the macOS app icon (1024 artwork on Apple's icon grid), favicon, and the desktop loading and failure screens.
7. **Hand-off format:** tokens as JSON or TS (Chakra-compatible names), component specs with sizes, and if possible HTML/React prototypes of the shell and of a card and an edge, so they can be built faithfully.

---

9. Open questions (for the owner, before or during the design)

- Dark mode: in scope? (Recommended: yes, designed together with light mode.)
- Is the toolbar too busy? Ideas welcome: move the relation filters onto the canvas, fold them into a menu on narrow windows, or add a command palette.
- Should the details panel get tabs (Overview · Relations · Code) instead of one long scroll?
- Does the product name "React App Explorer" stay?

---

10. Implementation plan (after the designs)

1. Tokens and theme (`packages/design-system/src/theme.ts`): colours, type, radii, shadows, light and dark.
2. Design-system components: restyle the existing ones, add any new ones (only this package touches Chakra).
3. The shell and toolbar (`App.tsx`, `Toolbar.tsx`, `Panel`), the file tree, the details panel.
4. The diagram (`canvas.css`, `CardShape`, `CardNode`, `FrameNode`, `ConditionEdge`, the tooltip). Measure on a 300-node canvas.
5. Brand assets: `icon.svg` → `bunx tauri icon`, favicon, loading page.
6. Check each step in both engines (Chromium on the dev server, WebKit against the desktop server) and with `capture-states.ts`; compare against the designs.

Appendix A. Today's design tokens (packages/design-system/src/theme.ts)

Chakra UI v3 `createSystem` config. Semantic tokens have light and dark values. Kind colours are `kind.<kind>.{solid,subtle,muted,fg}`; relation colours are `relation.<relation>`.

```
import { createSystem, defaultConfig, defineConfig } from "@chakra-ui/react";

/**
 * Design tokens for React App Explorer.
 * Graph node kinds and server/client boundaries get dedicated semantic tokens so
 * the app never hardcodes colors: use e.g. `bg="kind.component.subtle"`.
 * Blue is reserved for selection (selected / related nodes and edges): no kind or relation uses it.
 */
const kind = (base: string) => ({
  solid: { value: `{colors.${base}.500}` },
  subtle: { value: { _light: `{colors.${base}.50}`, _dark: `{colors.${base}.950}` } },
  muted: { value: { _light: `{colors.${base}.200}`, _dark: `{colors.${base}.800}` } },
  fg: { value: { _light: `{colors.${base}.700}`, _dark: `{colors.${base}.300}` } },
});

const config = defineConfig({
  globalCss: {
    "html, body, #root": { height: "100%", margin: 0 },
    // crisper type on macOS (both apps), Inter's single-storey a / open digits
    body: { fontFeatureSettings: "'cv11', 'ss01'", ...({ WebkitFontSmoothing: "antialiased",
MozOsxFontSmoothing: "grayscale" } as object) },
  },
  theme: {
    tokens: {
      colors: {
        // Components and their "renders" relation: #429cb5 at 500, tints and shades around it.
        component: {
          50: { value: "#eef8fb" },
          100: { value: "#d6eef4" },
          200: { value: "#b0deea" },
          300: { value: "#82c8da" },
          400: { value: "#5cb2c8" },
          500: { value: "#429cb5" },
          600: { value: "#367f94" },
          700: { value: "#2e6878" },
          800: { value: "#285563" },
          900: { value: "#234754" },
          950: { value: "#142c35" },
        },
      },
      // Softer, more modern corners across every component.
      radii: {
        l1: { value: "6px" },
        l2: { value: "8px" },
        l3: { value: "12px" },
        // the app shell's floating surfaces (file tree, canvas, details)
        panel: { value: "14px" },
      },
      shadows: {
        panel: { value: "0 1px 2px rgb(15 23 42 / 0.05), 0 10px 30px -16px rgb(15 23 42 / 0.18)" },
        control: { value: "0 1px 2px rgb(15 23 42 / 0.06)" },
      },
      fonts: {
        // bundled (@fontsource-variable, imported by the provider): the same in the browser and the app
        body: { value: "'Inter Variable', Inter, system-ui, -apple-system, Segoe UI, Roboto, sans-serif" },
        heading: { value: "'Inter Variable', Inter, system-ui, -apple-system, Segoe UI, Roboto, sans-serif" },
        mono: { value: "'JetBrains Mono Variable', 'JetBrains Mono', ui-monospace, SFMono-Regular, Menlo,
```

```
monospace" },
      },
    },
    semanticTokens: {
      colors: {
        // The semantic shades every Chakra palette has, so `colorPalette="component"` works (progress bars,
buttons…).
        component: {
          solid: { value: "{colors.component.500}" },
          contrast: { value: "white" },
          fg: { value: { _light: "{colors.component.700}", _dark: "{colors.component.300}" } },
          muted: { value: { _light: "{colors.component.100}", _dark: "{colors.component.900}" } },
          subtle: { value: { _light: "{colors.component.50}", _dark: "{colors.component.950}" } },
          emphasized: { value: { _light: "{colors.component.200}", _dark: "{colors.component.800}" } },
          focusRing: { value: "{colors.component.500}" },
        },
        kind: {
          module: kind("gray"),
          // Blue is reserved for selection; nothing else uses it.
          component: kind("component"),
          hook: kind("purple"),
          serverAction: kind("orange"),
          function: kind("teal"),
          type: kind("green"),
          route: kind("pink"),
          external: kind("yellow"),
        },
        boundary: {
          server: kind("gray"),
          client: kind("cyan"),
          shared: kind("pink"),
        },
        relation: {
          // "imports" edges: dark grey, distinct from the lighter module frames.
          imports: { value: { _light: "{colors.gray.700}", _dark: "{colors.gray.300}" } },
          importsMuted: { value: { _light: "{colors.gray.500}", _dark: "{colors.gray.500}" } },
          // Same colours as the edges and their labels on the canvas (sidebar badges reuse them).
          renders: { value: "{colors.component.500}" },
          usesHook: { value: "{colors.purple.500}" },
          invokes: { value: "{colors.teal.500}" },
          invokesAction: { value: "{colors.orange.500}" },
          declares: { value: "{colors.gray.500}" },
          usesType: { value: "{colors.green.600}" },
          navigates: { value: "{colors.pink.500}" },
          fetches: { value: "{colors.cyan.600}" },
          revalidates: { value: "{colors.gray.500}" },
          routeFile: { value: "{colors.pink.300}" },
          runsBefore: { value: "{colors.red.500}" },
          invalidates: { value: "{colors.purple.600}" },
        },
        // Code tokens (props, signatures, hooks) — shared by the node tooltip and the sidebar.
        // Note: no token may be called `value` — Chakra reads any object with a `value` key as a token.
        syntax: {
          name: { value: { _light: "{colors.gray.900}", _dark: "{colors.gray.50}" } },
          type: { value: { _light: "#0f766e", _dark: "{colors.teal.300}" } },
          literal: { value: { _light: "#b45309", _dark: "{colors.orange.300}" } },
          keyword: { value: { _light: "{colors.purple.600}", _dark: "{colors.purple.300}" } },
          punct: { value: { _light: "{colors.gray.400}", _dark: "{colors.gray.500}" } },
          bg: { value: { _light: "{colors.gray.50}", _dark: "{colors.gray.900}" } },
        },
        canvas: {
          bg: { value: { _light: "#FBFCFD", _dark: "{colors.gray.950}" } },
          dot: { value: { _light: "#D5DBE3", _dark: "{colors.gray.700}" } },
        },
        // the app shell: a cool grey backdrop the panels float on
        shell: {
          bg: { value: { _light: "#ECEFF3", _dark: "#0A0D12" } },
          border: { value: { _light: "#DDE2E8", _dark: "{colors.gray.800}" } },
        },
      },
```

```
    },
  },
});

export const system = createSystem(defaultConfig, config);

export type NodeKindToken = "module" | "component" | "hook" | "serverAction" | "function" | "type" | "route" |
"external";
export type BoundaryToken = "server" | "client" | "shared";
```

Appendix B. Today's diagram CSS (packages/viewer/src/components/canvas.css)

Cards, frames, edges, labels, the tooltip and the canvas chrome. Plain CSS on purpose: hundreds of nodes re-render while panning.

```
/*
 * Diagram styles — plain CSS on purpose: nodes and edges re-render on every drag frame, so they
 * avoid runtime CSS-in-JS. Colors come from the design-system tokens via Chakra's CSS variables.
 */

.rae-canvas {
  --sel: var(--chakra-colors-blue-500);
  --sel-soft: var(--chakra-colors-blue-300);
  --panel: var(--chakra-colors-bg-panel);
  --fg: var(--chakra-colors-fg);
  --fg-muted: var(--chakra-colors-fg-muted);
  --line: var(--chakra-colors-border);
  width: 100%;
  height: 100%;
  font-family: var(--chakra-fonts-body);
  color: var(--fg);
  outline: none;
}

/* Elements glide to a new placement — never while dragging (a frame's content would trail). */
.rae-canvas:not([data-dragging]) .react-flow__node {
  transition: transform 220ms ease, width 220ms ease, height 220ms ease, opacity 150ms ease;
}

/* Selection: everything not selected or related fades. */
.rae-canvas[data-selection] .react-flow__node:not(.selected):not(.rae-related) {
  opacity: 0.32;
}
.rae-canvas[data-selection] .react-flow__edge:not(.rae-edge--related) {
  opacity: 0.12;
}

/* ---- node kinds: colors ------------------------------------------------------------- */
[data-kind="module"] { --k: var(--chakra-colors-kind-module-solid); --k-soft: var(--chakra-colors-kind-module-
subtle); --k-mid: var(--chakra-colors-kind-module-muted); --k-fg: var(--chakra-colors-kind-module-fg); }
[data-kind="component"] { --k: var(--chakra-colors-kind-component-solid); --k-soft: var(--chakra-colors-kind-
component-subtle); --k-mid: var(--chakra-colors-kind-component-muted); --k-fg: var(--chakra-colors-kind-
component-fg); }
[data-kind="hook"] { --k: var(--chakra-colors-kind-hook-solid); --k-soft: var(--chakra-colors-kind-hook-
subtle); --k-mid: var(--chakra-colors-kind-hook-muted); --k-fg: var(--chakra-colors-kind-hook-fg); }
[data-kind="serverAction"] { --k: var(--chakra-colors-kind-server-action-solid); --k-soft: var(--chakra-colors-
kind-server-action-subtle); --k-mid: var(--chakra-colors-kind-server-action-muted); --k-fg: var(--chakra-
colors-kind-server-action-fg); }
[data-kind="function"] { --k: var(--chakra-colors-kind-function-solid); --k-soft: var(--chakra-colors-kind-
function-subtle); --k-mid: var(--chakra-colors-kind-function-muted); --k-fg: var(--chakra-colors-kind-function-
fg); }
[data-kind="type"] { --k: var(--chakra-colors-kind-type-solid); --k-soft: var(--chakra-colors-kind-type-
subtle); --k-mid: var(--chakra-colors-kind-type-muted); --k-fg: var(--chakra-colors-kind-type-fg); }
[data-kind="route"] { --k: var(--chakra-colors-kind-route-solid); --k-soft: var(--chakra-colors-kind-route-
subtle); --k-mid: var(--chakra-colors-kind-route-muted); --k-fg: var(--chakra-colors-kind-route-fg); }
[data-kind="external"] { --k: var(--chakra-colors-kind-external-solid); --k-soft: var(--chakra-colors-kind-
external-subtle); --k-mid: var(--chakra-colors-kind-external-muted); --k-fg: var(--chakra-colors-kind-external-
fg); }
[data-kind="folder"] { --k: var(--chakra-colors-fg-subtle); --k-soft: var(--chakra-colors-bg-subtle); --k-mid:
var(--chakra-colors-border-emphasized); --k-fg: var(--chakra-colors-fg-muted); }

/* ---- cards: one shape per kind ------------------------------------------------------ */
```

```
/* The shape is SVG (CardShape.tsx): tinted glass fill + soft sheen + outline. Modern leaf card:
   icon tile, name, quiet caption; actions float in on hover / selection. */
.rae-card {
  position: relative;
  width: 100%;
  height: 100%;
  --border: color-mix(in srgb, var(--k) 38%, transparent);
  --bw: 1.25px;
}
.react-flow__node.rae-related .rae-card { --border: var(--sel-soft); --bw: 2px; }
.react-flow__node.selected .rae-card { --border: var(--sel); --bw: 2px; }

.rae-card__svg {
  position: absolute;
  inset: 0;
  overflow: visible;
  /* elevation grows with nesting depth: elements float above their frames */
  /* static: hover doesn't recolour or re-filter the card (that read as blinking while the
     pointer crossed a diagram, and re-rasterised the shadow every transition frame) */
  filter: drop-shadow(0 calc(2px + var(--depth, 0) * 1.5px) calc(5px + var(--depth, 0) * 3px) rgb(15 23 42 /
0.12));
}
.react-flow__node.selected .rae-card__svg {
  filter: drop-shadow(0 calc(4px + var(--depth, 0) * 1.5px) calc(12px + var(--depth, 0) * 3px) rgb(59 130 246 /
0.38));
}
/* see-through glass, tinted towards the kind's colour in the top-left corner */
.rae-card__grad-a { stop-color: color-mix(in srgb, var(--k-soft) 92%, transparent); }
.rae-card__grad-b { stop-color: color-mix(in srgb, var(--panel) 62%, transparent); }
.rae-card__grad-c { stop-color: color-mix(in srgb, var(--panel) 52%, transparent); }
.rae-card__shine { stop-color: rgb(255 255 255 / 0.55); }
.rae-card__stroke { fill: none; stroke: var(--border); stroke-width: var(--bw); stroke-linejoin: round; }
.rae-card__stroke[data-dashed] { stroke-dasharray: 5 4; }
.rae-card__fold { stroke-width: 1.1px; }

.rae-card__body {
  position: relative;
  z-index: 1;
  height: 100%;
  display: flex;
  align-items: center;
  gap: 10px;
  padding: 0 12px 0 12px;
  min-width: 0;
}
/* text clear of the shapes' slanted / rounded ends */
.rae-card[data-kind="hook"] .rae-card__body { padding: 0 18px; }
.rae-card[data-kind="serverAction"] .rae-card__body { padding-right: 24px; }
.rae-card[data-kind="function"] .rae-card__body,
.rae-card[data-kind="external"] .rae-card__body { padding: 0 18px 0 14px; }
.rae-card[data-kind="module"] .rae-card__body { padding-right: 22px; }

.rae-card__tile {
  flex-shrink: 0;
  width: 32px;
  height: 32px;
  border-radius: 10px;
  display: inline-flex;
  align-items: center;
  justify-content: center;
  color: white;
  background: linear-gradient(135deg, color-mix(in srgb, var(--k) 82%, white), color-mix(in srgb, var(--k) 88%,
black));
  box-shadow: 0 2px 6px color-mix(in srgb, var(--k) 38%, transparent), inset 0 1px 0 rgb(255 255 255 / 0.35);
}
.rae-card[data-kind="function"] .rae-card__tile,
.rae-card[data-kind="external"] .rae-card__tile { border-radius: 999px; }
.rae-card[data-kind="folder"] .rae-card__tile { background: var(--chakra-colors-bg-muted); color: var(--chakra-
colors-fg-muted); box-shadow: inset 0 0 0 1px var(--chakra-colors-border); }
.rae-card[data-kind="external"] .rae-card__tile { background: color-mix(in srgb, var(--k) 18%, var(--panel));
```

```
color: var(--k-fg); box-shadow: inset 0 0 0 1px color-mix(in srgb, var(--k) 40%, transparent); }
.rae-card__text { min-width: 0; flex: 1; display: flex; flex-direction: column; gap: 1px; }
.rae-card .rae-name { font-size: 13.5px; font-weight: 600; letter-spacing: -0.01em; line-height: 18px; }
.rae-card__caption {
  font-size: 11px;
  line-height: 14px;
  color: var(--fg-muted);
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
}
.rae-card__boundary { flex-shrink: 0; align-self: flex-start; margin-top: 10px; transition: opacity 220ms
cubic-bezier(0.22, 1, 0.36, 1); }
/* the action toolbar takes its place on hover / selection */
.react-flow__node-card:hover .rae-card__boundary,
.react-flow__node-card.selected .rae-card__boundary { opacity: 0; }

/* hover: a subtle outline + wash in the node's own colour, faded in by opacity only (no filter
   or colour transitions, so nothing re-rasterises or blinks) */
.rae-card__hover {
  position: absolute;
  inset: 0;
  overflow: visible;
  pointer-events: none;
  opacity: 0;
  transition: opacity 260ms cubic-bezier(0.22, 1, 0.36, 1);
}
.rae-card__hover path {
  fill: color-mix(in srgb, var(--k) 7%, transparent);
  stroke: color-mix(in srgb, var(--k) 52%, transparent);
  stroke-width: 1.5px;
  stroke-linejoin: round;
}
.react-flow__node-card:hover .rae-card__hover { opacity: 1; }
/* selection keeps its blue outline on top */
.react-flow__node.selected .rae-card__hover path { stroke: none; }

/* actions: a small glass toolbar floating in on the right on hover / when selected */
.rae-card__actions {
  position: absolute;
  z-index: 2;
  right: 8px;
  top: 50%;
  transform: translate(4px, -50%);
  padding: 3px;
  border-radius: 10px;
  background: var(--panel);
  box-shadow: 0 4px 14px rgb(15 23 42 / 0.14), 0 0 0 1px var(--line);
  opacity: 0;
  pointer-events: none;
  transition: opacity 220ms cubic-bezier(0.22, 1, 0.36, 1), transform 220ms cubic-bezier(0.22, 1, 0.36, 1);
}
.react-flow__node-card:hover .rae-card__actions,
.react-flow__node-card:focus-within .rae-card__actions,
.react-flow__node-card.selected .rae-card__actions {
  opacity: 1;
  pointer-events: auto;
  transform: translate(0, -50%);
}
.rae-card__actions .rae-icon-btn { width: 22px; height: 22px; border-radius: 7px; }

/* ---- shared text, badges and chips (cards and frames) ---------------------------------- */
.rae-row { display: flex; align-items: center; gap: 6px; min-width: 0; }
.rae-icon { color: var(--k-fg); flex-shrink: 0; display: inline-flex; }
.rae-name {
  font-weight: 600;
  font-size: 13px;
  line-height: 18px;
  white-space: nowrap;
  overflow: hidden;
```

```
  text-overflow: ellipsis;
  min-width: 0;
}
.rae-sub {
  font-family: var(--chakra-fonts-mono);
  font-size: 10px;
  color: var(--fg-muted);
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
}
.rae-spacer { flex: 1; }
.rae-badges { display: flex; gap: 4px; flex: 1; min-width: 0; overflow: hidden; }
.rae-badge {
  font-size: 10px;
  line-height: 16px;
  padding: 0 5px;
  border-radius: 4px;
  white-space: nowrap;
  background: var(--k-soft);
  color: var(--k-fg);
  font-weight: 500;
}
.rae-badge--outline { background: transparent; color: var(--fg-muted); box-shadow: inset 0 0 0 1px var(--line);
}
.rae-chip {
  font-size: 10px;
  line-height: 16px;
  padding: 0 7px;
  border-radius: 999px;
  font-weight: 500;
  white-space: nowrap;
}
.rae-chip[data-boundary="server"] { background: var(--chakra-colors-boundary-server-subtle); color: var(--
chakra-colors-boundary-server-fg); box-shadow: inset 0 0 0 1px var(--chakra-colors-boundary-server-muted); }
.rae-chip[data-boundary="client"] { background: var(--chakra-colors-boundary-client-subtle); color: var(--
chakra-colors-boundary-client-fg); box-shadow: inset 0 0 0 1px var(--chakra-colors-boundary-client-muted); }
.rae-chip[data-boundary="shared"] { background: var(--chakra-colors-boundary-shared-subtle); color: var(--
chakra-colors-boundary-shared-fg); box-shadow: inset 0 0 0 1px var(--chakra-colors-boundary-shared-muted); }

/* ---- icon buttons ------------------------------------------------------------------ */
.rae-actions { display: flex; flex-shrink: 0; }
.rae-icon-btn {
  all: unset;
  box-sizing: border-box;
  width: 20px;
  height: 20px;
  display: inline-flex;
  align-items: center;
  justify-content: center;
  border-radius: 4px;
  color: var(--fg-muted);
  cursor: pointer;
}
.rae-icon-btn:hover { background: var(--chakra-colors-bg-muted); color: var(--fg); }
.rae-icon-btn:focus-visible { outline: 2px solid var(--sel); outline-offset: 1px; }
.rae-icon-btn:disabled { opacity: 0.35; cursor: default; background: none; }

/* ---- frames (file modules, folders) ------------------------------------------------ */
.rae-frame {
  position: relative;
  width: 100%;
  height: 100%;
  display: flex;
  flex-direction: column;
  border: 1.5px solid var(--k-mid);
  border-radius: 10px;
  /* translucent: nested frames stack their tint (deeper = denser), what's behind shows through */
  background: color-mix(in srgb, var(--k-soft) 30%, transparent);
  /* layered shadow, bigger the deeper the frame sits: each level reads as a raised sheet */
```

```
  box-shadow:
    inset 0 1px 0 rgb(255 255 255 / 0.45),
    0 1px 2px rgb(15 23 42 / 0.06),
    0 calc(4px + var(--depth, 0) * 4px) calc(12px + var(--depth, 0) * 8px) rgb(15 23 42 / 0.09);
}
.rae-frame[data-kind="folder"] {
  border-style: dashed;
  background: color-mix(in srgb, var(--k-soft) 22%, transparent);
  border-radius: 4px 12px 12px 12px;
  box-shadow: 0 calc(2px + var(--depth, 0) * 3px) calc(10px + var(--depth, 0) * 6px) rgb(15 23 42 / 0.06);
}
.react-flow__node.rae-related > .rae-frame { border-color: var(--sel-soft); }
.react-flow__node.selected > .rae-frame { border-color: var(--sel); border-width: 2.5px; box-shadow: 0 0 0 4px
rgb(59 130 246 / 0.15); }
.rae-frame[data-refused] { border: 2px dashed var(--chakra-colors-red-solid); background: color-mix(in srgb,
var(--chakra-colors-red-subtle) 50%, transparent); }

/* frame hover: same idea, in the file's / folder's colour */
.rae-frame::after {
  content: "";
  position: absolute;
  inset: -1.5px;
  border-radius: inherit;
  border: 1.5px solid color-mix(in srgb, var(--k) 55%, transparent);
  background: color-mix(in srgb, var(--k) 4%, transparent);
  pointer-events: none;
  opacity: 0;
  transition: opacity 260ms cubic-bezier(0.22, 1, 0.36, 1);
}
.rae-frame[data-kind="folder"]::after { border-style: dashed; }
/* frames (files holding elements, folders): hover only on the header — where they're selected and
   dragged — never over their content */
.rae-frame:has(> .rae-frame__header:hover)::after { opacity: 1; }
.rae-frame__header { transition: background-color 260ms cubic-bezier(0.22, 1, 0.36, 1); }
.rae-frame[data-kind="folder"] .rae-frame__header { border-radius: 3px 11px 0 0; }
.rae-frame__header:hover { background: color-mix(in srgb, var(--k) 8%, var(--panel)); }
.rae-frame[data-kind="folder"] .rae-frame__header:hover { background: color-mix(in srgb, var(--k) 7%,
transparent); }
.react-flow__node.selected > .rae-frame::after,
.rae-frame[data-refused]::after { border-color: transparent; }

.rae-frame__header {
  height: 48px;
  flex-shrink: 0;
  display: flex;
  align-items: center;
  gap: 8px;
  padding: 0 10px;
  border-bottom: 1px solid var(--k-mid);
  background: color-mix(in srgb, var(--panel) 50%, transparent);
  border-radius: 9px 9px 0 0;
  cursor: grab;
}
.rae-frame[data-kind="folder"] .rae-frame__header { background: transparent; border-bottom-style: dashed; }
/* folder tab */
.rae-frame[data-kind="folder"]::before {
  content: "";
  position: absolute;
  left: -1.5px;
  top: -9px;
  width: 64px;
  height: 9px;
  border: 1.5px dashed var(--k-mid);
  border-bottom: none;
  border-radius: 6px 6px 0 0;
}
.rae-frame__title { min-width: 0; flex: 1; }
.rae-frame__room { flex: 1; }

/* ---- edges: every relation has its own colour, dash shape and speed ------------------- */
```

```
.react-flow__edge.rae-edge path.react-flow__edge-path {
  stroke: var(--e);
  stroke-width: var(--ew, 1.5px);
  stroke-dasharray: var(--dash);
  stroke-linecap: round;
  animation: var(--anim) var(--speed) linear infinite;
}
.rae-edge--imports        { --e: var(--chakra-colors-relation-imports); --dash: 8 6; --anim: rae-dash-14; --
speed: 1.8s; }
.rae-edge--imports-lazy   { --e: var(--chakra-colors-relation-imports); --dash: 3 6; --anim: rae-dash-9; --
speed: 2.4s; }
.rae-edge--imports-type   { --e: var(--chakra-colors-relation-imports-muted); --dash: 1 5; --anim: rae-dash-6;
--speed: 3.2s; --ew: 1.25px; }
.rae-edge--renders        { --e: var(--chakra-colors-kind-component-solid); --dash: 12 5; --anim: rae-dash-17;
--speed: 0.9s; --ew: 2px; }
.rae-edge--usesHook       { --e: var(--chakra-colors-kind-hook-solid); --dash: 2 5; --anim: rae-dash-7; --
speed: 1.3s; --ew: 2px; }
.rae-edge--usesType       { --e: var(--chakra-colors-relation-uses-type); --dash: 1 4; --anim: rae-dash-5; --
speed: 1.8s; --ew: 1.5px; }
.rae-edge--navigates      { --e: var(--chakra-colors-relation-navigates); --dash: 10 4; --anim: rae-dash-14; --
speed: 0.9s; --ew: 2.25px; }
.rae-edge--fetches        { --e: var(--chakra-colors-relation-fetches); --dash: 6 4; --anim: rae-dash-9; --
speed: 1s; --ew: 2px; }
.rae-edge--revalidates    { --e: var(--chakra-colors-relation-revalidates); --dash: 1 4; --anim: rae-dash-5; --
speed: 2s; --ew: 1.5px; }
.rae-edge--runsBefore     { --e: var(--chakra-colors-relation-runs-before); --dash: 2 3; --anim: rae-dash-5; --
speed: 1.2s; --ew: 2px; }
.rae-edge--invalidates    { --e: var(--chakra-colors-relation-invalidates); --dash: 4 3; --anim: rae-dash-7; --
speed: 1.4s; --ew: 1.75px; }
.rae-edge--routeFile      { --e: var(--chakra-colors-relation-route-file); --dash: 3 4; --anim: rae-dash-7; --
speed: 2.4s; --ew: 1.5px; }
.rae-edge--invokes        { --e: var(--chakra-colors-kind-function-solid); --dash: 10 3 2 3; --anim: rae-dash-
18; --speed: 0.7s; --ew: 1.75px; }
.rae-edge--invokesAction  { --e: var(--chakra-colors-kind-server-action-solid); --dash: 6 3; --anim: rae-dash-
9; --speed: 0.45s; --ew: 2.25px; }
.rae-edge--declares       { --e: var(--chakra-colors-kind-module-muted); --dash: 2 4; --anim: rae-dash-6; --
speed: 2.5s; }
/* The selection's edges keep their relation colour, drawn thicker. */
.react-flow__edge.rae-edge--related path.react-flow__edge-path { --ew: 3px; }

@keyframes rae-dash-5 { to { stroke-dashoffset: -5; } }
@keyframes rae-dash-6 { to { stroke-dashoffset: -6; } }
@keyframes rae-dash-7 { to { stroke-dashoffset: -7; } }
@keyframes rae-dash-9 { to { stroke-dashoffset: -9; } }
@keyframes rae-dash-14 { to { stroke-dashoffset: -14; } }
@keyframes rae-dash-17 { to { stroke-dashoffset: -17; } }
@keyframes rae-dash-18 { to { stroke-dashoffset: -18; } }

/* Edges hold still while something is dragged, so the frames stay smooth. */
.rae-canvas[data-dragging] .react-flow__edge-path { animation-play-state: paused; }
.rae-canvas[data-still] .react-flow__edge-path { animation: none !important; }
@media (prefers-reduced-motion: reduce) {
  .react-flow__edge-path { animation: none !important; }
  .rae-canvas .react-flow__node { transition: none !important; }
}

/* Relation labels, in the relation's colour. */
.rae-edge-label {
  position: absolute;
  pointer-events: all;
  --e: var(--chakra-colors-relation-imports);
  /* above every edge layer (React Flow lifts edges of selected nodes to z 1000+), opaque so no
     line runs through the text */
  z-index: 1100;
  background: var(--panel);
  border: 1px solid color-mix(in srgb, var(--e) 55%, var(--panel));
  border-radius: 999px;
  padding: 0 7px;
  font: 10px/16px var(--chakra-fonts-body);
  color: var(--e);
```

```
  max-width: 240px;
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
  cursor: help;
}
.rae-edge-label b { font-weight: 600; }
.rae-edge-label__when { font-family: var(--chakra-fonts-mono); color: var(--fg-muted); }
.rae-edge-label[data-kind="renders"] { --e: var(--chakra-colors-kind-component-solid); }
.rae-edge-label[data-kind="usesHook"] { --e: var(--chakra-colors-kind-hook-solid); }
.rae-edge-label[data-kind="usesType"] { --e: var(--chakra-colors-relation-uses-type); }
.rae-edge-label[data-kind="navigates"] { --e: var(--chakra-colors-relation-navigates); }
.rae-edge-label[data-kind="fetches"] { --e: var(--chakra-colors-relation-fetches); }
.rae-edge-label[data-kind="revalidates"] { --e: var(--chakra-colors-relation-revalidates); }
.rae-edge-label[data-kind="runsBefore"] { --e: var(--chakra-colors-relation-runs-before); }
.rae-edge-label[data-kind="invalidates"] { --e: var(--chakra-colors-relation-invalidates); }
.rae-edge-label[data-kind="routeFile"], .rae-edge-label[data-kind="routeChild"] { --e: var(--chakra-colors-
relation-route-file); }
.rae-edge-label[data-kind="invokes"] { --e: var(--chakra-colors-kind-function-solid); }
.rae-edge-label[data-kind="invokesAction"] { --e: var(--chakra-colors-kind-server-action-solid); }
.rae-edge-label[data-related] { border-width: 1.5px; border-color: var(--e); font-weight: 600; }
.rae-canvas[data-selection] .rae-edge-label:not([data-related]) { opacity: 0.15; }

.rae-legend {
  position: absolute;
  left: 56px;
  bottom: 12px;
  z-index: 5;
}

/* ---- cursors: what a press would do ------------------------------------------------- */
.rae-canvas .react-flow__pane { cursor: grab; }
.rae-canvas .react-flow__pane.dragging { cursor: grabbing; }
.rae-canvas .react-flow__node-card { cursor: grab; }                /* drag to move, click to select */
.rae-canvas .react-flow__node-card:not(.draggable) { cursor: pointer; } /* a file's only element: auto-placed,
click only */
.rae-canvas .rae-frame__header { cursor: grab; }                     /* frames move by their header */
.rae-canvas .rae-frame__room { cursor: grab; }                       /* the room pans, like the canvas */
.rae-canvas .rae-frame__room[data-panning] { cursor: grabbing; }
.rae-canvas .rae-icon-btn:not(:disabled) { cursor: pointer; }
.rae-canvas .rae-icon-btn:disabled { cursor: not-allowed; }
.rae-canvas[data-adding] .react-flow__node-card,
.rae-canvas[data-adding] .rae-frame__header { cursor: copy; }        /* ⌘/Ctrl held: click adds to the
selection */
.rae-canvas[data-dragging],
.rae-canvas[data-dragging] * { cursor: grabbing !important; }
.rae-canvas[data-dragging][data-refused-drop],
.rae-canvas[data-dragging][data-refused-drop] * { cursor: not-allowed !important; } /* it won't go in there */
.rae-canvas .react-flow__controls-button { cursor: pointer; }

/* ---- node tooltip (one for the whole canvas) — same language as the sidebar ------------- */
.rae-tip {
  position: fixed;
  z-index: 1500;
  pointer-events: none;
  width: max-content;
  min-width: 240px;
  max-width: 380px;
  background: color-mix(in srgb, var(--chakra-colors-bg-panel) 96%, transparent);
  color: var(--chakra-colors-fg);
  border: 1px solid var(--chakra-colors-border-muted);
  border-radius: 14px;
  box-shadow: 0 12px 32px rgb(15 23 42 / 0.16), 0 2px 6px rgb(15 23 42 / 0.06);
  font: 12px/1.45 var(--chakra-fonts-body);
  animation: rae-tip-in 180ms cubic-bezier(0.2, 0.8, 0.2, 1);
}
/* accent line in the kind's colour */
.rae-tip::before {
  content: "";
  position: absolute;
```

```
  left: 14px;
  right: 14px;
  top: -1px;
  height: 3px;
  border-radius: 0 0 3px 3px;
  background: linear-gradient(90deg, var(--k), color-mix(in srgb, var(--k) 30%, transparent));
}
/* arrow pointing at the node */
.rae-tip::after {
  content: "";
  position: absolute;
  left: 50%;
  width: 10px;
  height: 10px;
  background: inherit;
  border: inherit;
  border-top: none;
  border-left: none;
  transform: translateX(-50%) rotate(45deg);
}
.rae-tip[data-side="top"]::after { bottom: -6px; }
.rae-tip[data-side="bottom"]::after { top: -6px; transform: translateX(-50%) rotate(-135deg); }
@keyframes rae-tip-in {
  from { opacity: 0; margin-top: 4px; }
  to { opacity: 1; margin-top: 0; }
}
.rae-tip__head { display: flex; align-items: center; gap: 10px; padding: 12px 14px 0; }
.rae-tip__tile { width: 30px; height: 30px; border-radius: 9px; }
.rae-tip__titles { min-width: 0; }
.rae-tip__title { font-weight: 600; font-size: 14px; letter-spacing: -0.01em; line-height: 18px; }
.rae-tip__caption { color: var(--chakra-colors-fg-muted); font-size: 11px; }
.rae-tip__path {
  margin: 8px 14px 0;
  padding: 3px 8px;
  border-radius: 6px;
  background: var(--chakra-colors-syntax-bg);
  font: 11px/1.5 var(--chakra-fonts-mono);
  color: var(--chakra-colors-fg-muted);
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
}
.rae-tip__body { padding: 4px 14px 12px; }
.rae-tip__section { margin-top: 9px; }
.rae-tip__label { font-size: 10px; font-weight: 600; letter-spacing: 0.06em; text-transform: uppercase; color:
var(--chakra-colors-fg-subtle); margin-bottom: 4px; }
.rae-tip__code { font: 11.5px/1.6 var(--chakra-fonts-mono); }
.rae-tip__code > div { white-space: nowrap; overflow: hidden; text-overflow: ellipsis; }
.rae-tip__empty { color: var(--chakra-colors-fg-muted); font-size: 11.5px; }
.rae-tip__foot { margin-top: 9px; color: var(--chakra-colors-fg-muted); font-size: 11px; }
.rae-tip__more { font-size: 11px; color: var(--chakra-colors-fg-subtle); }
.rae-tip__chips { display: flex; flex-wrap: wrap; gap: 4px; align-items: center; }
.rae-tip__chip {
  display: inline-flex;
  align-items: center;
  height: 20px;
  padding: 0 8px;
  border-radius: 999px;
  font: 500 11px/1 var(--chakra-fonts-mono);
  color: color-mix(in srgb, var(--c) 82%, var(--chakra-colors-fg));
  background: color-mix(in srgb, var(--c) 11%, transparent);
  box-shadow: inset 0 0 0 1px color-mix(in srgb, var(--c) 30%, transparent);
}
/* code tokens — the same colours the sidebar uses (code.* tokens) */
.rae-tok--name { color: var(--chakra-colors-syntax-name); font-weight: 500; }
.rae-tok--type { color: var(--chakra-colors-syntax-type); }
.rae-tok--value { color: var(--chakra-colors-syntax-literal); }
.rae-tok--punct { color: var(--chakra-colors-syntax-punct); }
.rae-tok--keyword { color: var(--chakra-colors-syntax-keyword); }
```

```
/* The round "+" on a file's bottom border: what it contains that isn't drawn. */
.rae-contains {
  all: unset;
  box-sizing: border-box;
  position: absolute;
  left: 50%;
  bottom: 0;
  transform: translate(-50%, 50%);
  min-width: 26px;
  height: 20px;
  padding: 0 6px;
  border-radius: 999px;
  display: inline-flex;
  align-items: center;
  justify-content: center;
  font: 600 10px/1 var(--chakra-fonts-body);
  background: var(--chakra-colors-bg-panel);
  color: var(--chakra-colors-kind-module-fg);
  border: 1.5px solid var(--chakra-colors-kind-module-muted);
  box-shadow: 0 1px 3px rgb(0 0 0 / 0.12);
  z-index: 3;
  cursor: pointer;
}
.rae-contains:hover { border-color: var(--chakra-colors-kind-module-solid); }
.rae-contains:focus-visible { outline: 2px solid var(--sel); outline-offset: 1px; }

/* An imported project file that doesn't exist: faded, dashed outline (CardShape) */
.rae-card[data-missing] .rae-card__body { opacity: 0.6; }

/* type definitions in the node tooltip: keep the source layout */
.rae-tip__pre { white-space: pre-wrap; word-break: break-word; margin: 0; max-height: 260px; overflow: hidden;
}

/* route frames: the route's colour on the header */
.rae-frame[data-kind="route"] .rae-frame__header { background: color-mix(in srgb, var(--k) 8%, transparent); }

/* route rendering badges, as `next build` labels them */
.rae-chip[data-render="static"] { --c: var(--chakra-colors-green-solid); }
.rae-chip[data-render="ssg"] { --c: var(--chakra-colors-teal-solid); }
.rae-chip[data-render="dynamic"] { --c: var(--chakra-colors-orange-solid); }
.rae-chip[data-render="ppr"] { --c: var(--chakra-colors-purple-solid); }
.rae-chip[data-render="blocking"] { --c: var(--chakra-colors-red-solid); }
.rae-chip[data-cache] { --c: var(--chakra-colors-purple-solid); color: var(--c); background: color-mix(in srgb,
var(--c) 12%, transparent); border-color: color-mix(in srgb, var(--c) 30%, transparent); white-space: nowrap; }
.rae-chip[data-render] { color: var(--c); background: color-mix(in srgb, var(--c) 12%, transparent); border-
color: color-mix(in srgb, var(--c) 30%, transparent); font-variant-numeric: tabular-nums; white-space: nowrap;
}

/* WebKit (the desktop app's web view): no hover fades or slides on nodes. An animating opacity /
   transform gets its own compositing layer, rasterised at 1× and then scaled by the diagram's zoom,
   so the card read blurry until the animation ended. Instant changes repaint at the real scale. */
:root[data-webkit] .rae-card__hover,
:root[data-webkit] .rae-card__actions,
:root[data-webkit] .rae-card__boundary,
:root[data-webkit] .rae-frame::after,
:root[data-webkit] .rae-frame__header {
  transition: none;
}
:root[data-webkit] .rae-card__actions { transform: translate(0, -50%); }

/* ---- canvas chrome: zoom controls and minimap as small floating surfaces ------------------ */
.rae-canvas .react-flow__controls {
  box-shadow: 0 1px 2px rgb(15 23 42 / 0.06), 0 6px 16px -8px rgb(15 23 42 / 0.18);
  border: 1px solid var(--chakra-colors-shell-border);
  border-radius: 12px;
  overflow: hidden;
  background: var(--chakra-colors-bg-panel);
  padding: 3px;
  gap: 2px;
}
```

```
.rae-canvas .react-flow__controls-button {
  border: 0;
  border-radius: 8px;
  width: 28px;
  height: 28px;
  background: transparent;
  color: var(--chakra-colors-fg-muted);
}
.rae-canvas .react-flow__controls-button:hover:not(:disabled) { background: var(--chakra-colors-bg-muted);
color: var(--chakra-colors-fg); }
.rae-canvas .react-flow__controls-button:disabled { opacity: 0.35; }
.rae-canvas .react-flow__controls-button svg { fill: currentColor; max-width: 13px; max-height: 13px; }
.rae-canvas .react-flow__minimap {
  border: 1px solid var(--chakra-colors-shell-border);
  border-radius: 12px;
  overflow: hidden;
  box-shadow: 0 1px 2px rgb(15 23 42 / 0.06), 0 6px 16px -8px rgb(15 23 42 / 0.18);
  background: var(--chakra-colors-bg-panel);
}
```

Appendix C. Logo and app-icon artwork (packages/desktop/icon.svg)



```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 1024 1024">
  <defs>
    <linearGradient id="bg" x1="0.15" y1="0" x2="0.85" y2="1">
      <stop offset="0" stop-color="#1D4E63"/>
      <stop offset="0.55" stop-color="#14304A"/>
      <stop offset="1" stop-color="#0B1426"/>
    </linearGradient>
    <radialGradient id="glow" cx="0.5" cy="0.46" r="0.5">
      <stop offset="0" stop-color="#7DD3E8" stop-opacity="0.45"/>
      <stop offset="1" stop-color="#7DD3E8" stop-opacity="0"/>
    </radialGradient>
    <linearGradient id="sheen" x1="0" y1="0" x2="0" y2="1">
      <stop offset="0" stop-color="#FFFFFF" stop-opacity="0.16"/>
      <stop offset="0.5" stop-color="#FFFFFF" stop-opacity="0"/>
    </linearGradient>
    <linearGradient id="core" x1="0" y1="0" x2="1" y2="1">
      <stop offset="0" stop-color="#FFFFFF"/>
      <stop offset="1" stop-color="#D6EEF4"/>
    </linearGradient>
    <linearGradient id="teal" x1="0" y1="0" x2="1" y2="1"><stop offset="0" stop-color="#82C8DA"/><stop
offset="1" stop-color="#429CB5"/></linearGradient>
    <linearGradient id="violet" x1="0" y1="0" x2="1" y2="1"><stop offset="0" stop-color="#C4B5FD"/><stop
offset="1" stop-color="#8B5CF6"/></linearGradient>
    <linearGradient id="amber" x1="0" y1="0" x2="1" y2="1"><stop offset="0" stop-color="#FDBA74"/><stop
offset="1" stop-color="#F97316"/></linearGradient>
    <filter id="shadow" x="-20%" y="-20%" width="140%" height="140%"><feDropShadow dx="0" dy="10"
stdDeviation="14" flood-color="#020617" flood-opacity="0.45"/></filter>
  </defs>
  <!-- macOS icon grid: 824px body on a 1024 canvas -->
  <g filter="url(#shadow)">
    <rect x="100" y="100" width="824" height="824" rx="186" fill="url(#bg)"/>
  </g>
  <rect x="100" y="100" width="824" height="824" rx="186" fill="url(#glow)"/>
  <rect x="100" y="100" width="824" height="824" rx="186" fill="url(#sheen)"/>
  <rect x="101.5" y="101.5" width="821" height="821" rx="184.5" fill="none" stroke="#FFFFFF" stroke-
opacity="0.12" stroke-width="3"/>
  <!-- orbit -->
  <circle cx="512" cy="500" r="196" fill="none" stroke="#FFFFFF" stroke-opacity="0.18" stroke-width="10"
stroke-dasharray="2 30" stroke-linecap="round"/>
  <!-- relations -->
  <g stroke="#FFFFFF" stroke-opacity="0.55" stroke-width="22" stroke-linecap="round">
    <line x1="512" y1="500" x2="318" y2="322"/>
    <line x1="512" y1="500" x2="716" y2="330"/>
    <line x1="512" y1="500" x2="600" y2="732"/>
  </g>
  <!-- elements: component, hook, server action -->
  <rect x="244" y="248" width="148" height="148" rx="40" fill="url(#teal)"/>
  <rect x="648" y="262" width="136" height="136" rx="68" fill="url(#violet)"/>
  <rect x="536" y="668" width="128" height="128" rx="34" fill="url(#amber)" transform="rotate(12 600 732)"/>
  <!-- the explored node -->
  <circle cx="512" cy="500" r="104" fill="url(#core)"/>
  <circle cx="512" cy="500" r="52" fill="#14304A"/>
```

```
  <circle cx="512" cy="500" r="22" fill="#7DD3E8"/>
</svg>
```

Appendix D. The data: one node and one relation of each kind

From the scan of the demo shop the screenshots show (145 nodes, 330 relations). Every label, badge and section in the UI comes from these fields.


```
{
  "framework": "next-app",
  "next": {
    "cacheComponents": true
  },
  "stats": {
    "files": 45,
    "durationMs": 212,
    "phases": {
      "discover": 41.8,
      "cacheLoad": 0,
      "parse": 14.5,
      "graph": 4.5,
      "types": 149.1
    },
    "cacheHits": 0,
    "types": true
  },
  "nodes": {
    "module": {
      "id": "app/(marketing)/about/page.tsx",
      "kind": "module",
      "name": "page.tsx",
      "filePath": "app/(marketing)/about/page.tsx",
      "absPath": "/Users/me/IdeaProjects/next-shop-demo/app/(marketing)/about/page.tsx",
      "loc": {
        "line": 1,
        "col": 1
      },
      "nextRole": "page",
      "moduleType": "server"
    },
    "component": {
      "id": "app/(marketing)/about/page.tsx#AboutPage",
      "kind": "component",
      "name": "AboutPage",
      "filePath": "app/(marketing)/about/page.tsx",
      "absPath": "/Users/me/IdeaProjects/next-shop-demo/app/(marketing)/about/page.tsx",
      "loc": {
        "line": 1,
        "col": 25
      },
      "moduleId": "app/(marketing)/about/page.tsx",
      "exported": true,
      "exportNames": [
        "default"
      ],
      "nextRole": "page",
      "signature": {
        "params": [],
        "returnType": "Element"
      },
      "moduleType": "server",
      "renderTree": "(1 JSX elements, trimmed)"
    },
    "function": {
      "id": "app/(shop)/@modal/default.tsx#NoModal",
      "kind": "function",
      "name": "NoModal",
```

```
      "filePath": "app/(shop)/@modal/default.tsx",
      "absPath": "/Users/me/IdeaProjects/next-shop-demo/app/(shop)/@modal/default.tsx",
      "loc": {
        "line": 1,
        "col": 25
      },
      "moduleId": "app/(shop)/@modal/default.tsx",
      "exported": true,
      "exportNames": [
        "default"
      ],
      "nextRole": "default",
      "signature": {
        "params": [],
        "returnType": "null"
      },
      "moduleType": "server"
    },
    "type": {
      "id": "app/api/products/[id]/route.ts#Ctx",
      "kind": "type",
      "name": "Ctx",
      "filePath": "app/api/products/[id]/route.ts",
      "absPath": "/Users/me/IdeaProjects/next-shop-demo/app/api/products/[id]/route.ts",
      "loc": {
        "line": 4,
        "col": 6
      },
      "moduleId": "app/api/products/[id]/route.ts",
      "exported": false,
      "typeText": "type Ctx = { params: Promise<{ id: string }> };",
      "typeKind": "type",
      "moduleType": "server"
    },
    "hook": {
      "id": "hooks/useCart.tsx#useCart",
      "kind": "hook",
      "name": "useCart",
      "filePath": "hooks/useCart.tsx",
      "absPath": "/Users/me/IdeaProjects/next-shop-demo/hooks/useCart.tsx",
      "loc": {
        "line": 23,
        "col": 17
      },
      "moduleId": "hooks/useCart.tsx",
      "exported": true,
      "exportNames": [
        "useCart"
      ],
      "signature": {
        "params": [],
        "returnType": "CartState"
      },
      "typeRefs": {
        "CartState": "hooks/useCart.tsx#CartState"
      },
      "hooksUsed": [
        {
          "hook": "useContext",
          "targetId": "ext:react",
          "args": [
            "CartContext"
          ],
          "assignedTo": "cart",
          "returnType": "CartState | null",
          "loc": {
            "line": 24,
            "col": 16
          }
        }
```

```
      ],
      "moduleType": "client"
    },
    "serverAction": {
      "id": "lib/actions.ts#addToCart",
      "kind": "serverAction",
      "name": "addToCart",
      "filePath": "lib/actions.ts",
      "absPath": "/Users/me/IdeaProjects/next-shop-demo/lib/actions.ts",
      "loc": {
        "line": 8,
        "col": 23
      },
      "moduleId": "lib/actions.ts",
      "exported": true,
      "exportNames": [
        "addToCart"
      ],
      "isAsync": true,
      "signature": {
        "params": [
          {
            "name": "item",
            "type": "CartItem",
            "optional": false
          }
        ],
        "returnType": "Promise<void>"
      },
      "dynamicApis": [
        "cookies()"
      ],
      "moduleType": "server",
      "navigations": [
        {
          "via": "revalidate",
          "href": "/cart",
          "route": "route:/(shop)/cart"
        }
      ]
    },
    "external": {
      "id": "ext:react",
      "kind": "external",
      "name": "react",
      "filePath": "react"
    },
    "route": {
      "id": "route:/",
      "kind": "route",
      "name": "/",
      "filePath": "app",
      "absPath": "/Users/me/IdeaProjects/next-shop-demo/app",
      "route": {
        "url": "/",
        "segment": "",
        "segmentKind": "root",
        "params": [],
        "kind": "segment",
        "files": {
          "layout": "app/layout.tsx",
          "loading": "app/loading.tsx",
          "not-found": "app/not-found.tsx"
        }
      }
    }
  },
  "edges": [
    {
      "from": "app/(marketing)/about/page.tsx",
```

```
      "to": "app/(marketing)/about/page.tsx#AboutPage",
      "kind": "declares"
    },
    {
      "from": "app/(marketing)/layout.tsx",
      "to": "ext:react",
      "kind": "imports",
      "importType": "type-only",
      "specifiers": [
        "ReactNode"
      ]
    },
    {
      "from": "app/(marketing)/layout.tsx#MarketingLayout",
      "to": "components/Header.tsx#Header",
      "kind": "renders"
    },
    {
      "from": "app/(marketing)/page.tsx#HomePage",
      "to": "lib/data.ts#getProducts",
      "kind": "invokes"
    },
    {
      "from": "app/(shop)/cart/page.tsx#CartPage",
      "to": "hooks/useCart.tsx#useCart",
      "kind": "usesHook",
      "specifiers": [
        "useCart"
      ]
    },
    {
      "from": "app/(shop)/checkout/page.tsx#CheckoutPage",
      "to": "lib/actions.ts#placeOrder",
      "kind": "invokesAction"
    },
    {
      "from": "app/api/products/[id]/route.ts#GET",
      "to": "app/api/products/[id]/route.ts#Ctx",
      "kind": "usesType"
    },
    {
      "from": "app/api/revalidate/route.ts#POST",
      "to": "lib/catalog.ts#getDeals",
      "kind": "invalidates",
      "specifiers": [
        "deals"
      ]
    },
    {
      "from": "route:/",
      "to": "app/layout.tsx",
      "kind": "routeFile"
    },
    {
      "from": "route:/",
      "to": "route:/(marketing)",
      "kind": "routeChild"
    },
    {
      "from": "proxy.ts",
      "to": "route:/dashboard/settings",
      "kind": "runsBefore"
    },
    {
      "from": "app/(marketing)/page.tsx#HomePage",
      "to": "route:/(shop)/products",
      "kind": "navigates",
      "href": "/products",
      "via": "link"
    },
```

```
    {
      "from": "app/(shop)/cart/page.tsx#CartPage",
      "to": "route:/api/products",
      "kind": "fetches",
      "href": "/api/products",
      "via": "fetch"
    },
    {
      "from": "app/api/revalidate/route.ts#POST",
      "to": "route:/(shop)/products",
      "kind": "revalidates",
      "href": "/products",
      "via": "revalidate"
    }
  ]
}
```